

ROBOTICS

# KAN ROBOT CONTROL

By Michela Faella



# INTRODUCTION



# INTRODUCTION

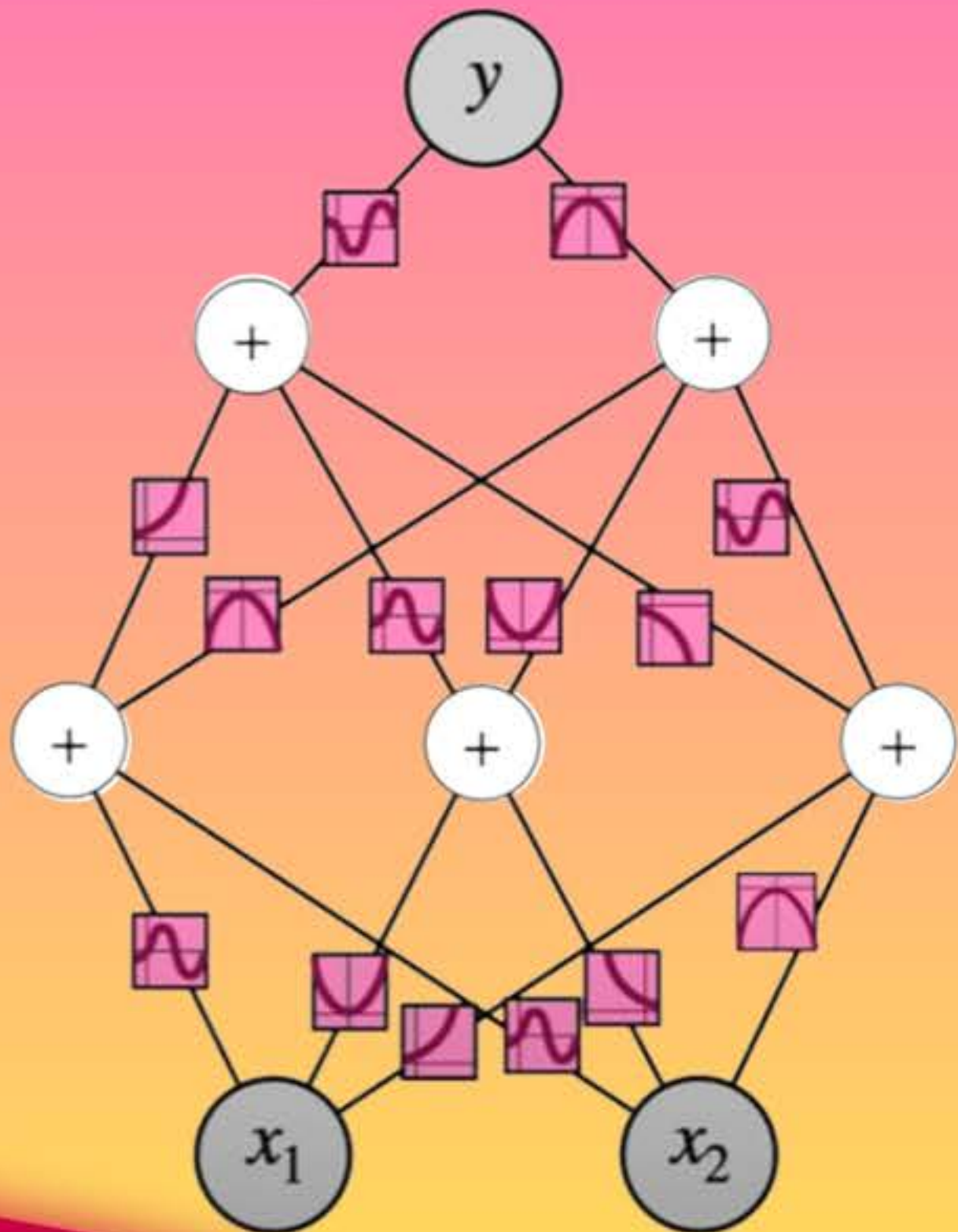
## Project Definition

This work investigates the performance of **Kolmogorov-Arnold Networks** (KAN) compared to standard **Multi-Layer Perceptrons** (MLP) for estimating the final 3D position of markers on a soft robotic system undergoing mechanical deformation. Both models are trained on static actuation data and evaluated on unseen configurations, with a focus on generalization across actuation subspaces. As an **extension**, we explore a **sequential variant of KAN** (KAN-RNN), incorporating temporal dynamics through recurrent processing. Experimental results highlight the superior accuracy of KAN over MLP in the static setting, while the KAN-RNN shows potential but remains less competitive in its current form.





# KOLMOGOROV-ARNOLD NETWORKS



## Definition

Kolmogorov–Arnold Network (KAN) is a novel **neural network architecture** in which neural weights are replaced by **learnable univariate functions**.

## General Idea

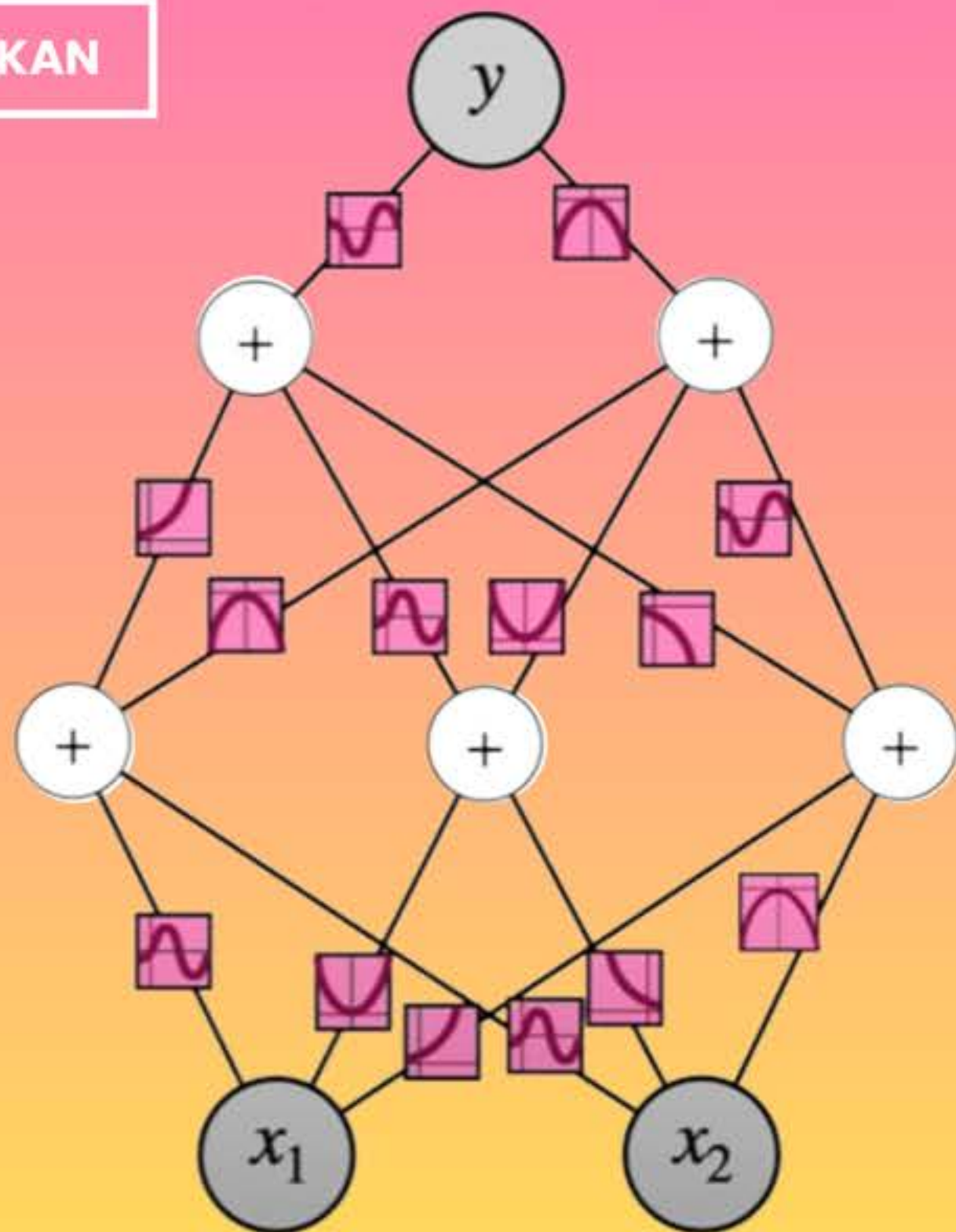
- On the regular NN, we have **fixed activation functions** (RELU, Sigmoid, TanH) and we **train the weights**.
- On a KAN, the **weights are fixed** and the we **train the activation functions**.

*"By replacing weights with learnable functions and leveraging the Kolmogorov–Arnold representation theorem, KANs provide a structurally different and often more interpretable alternative to traditional neural networks."*

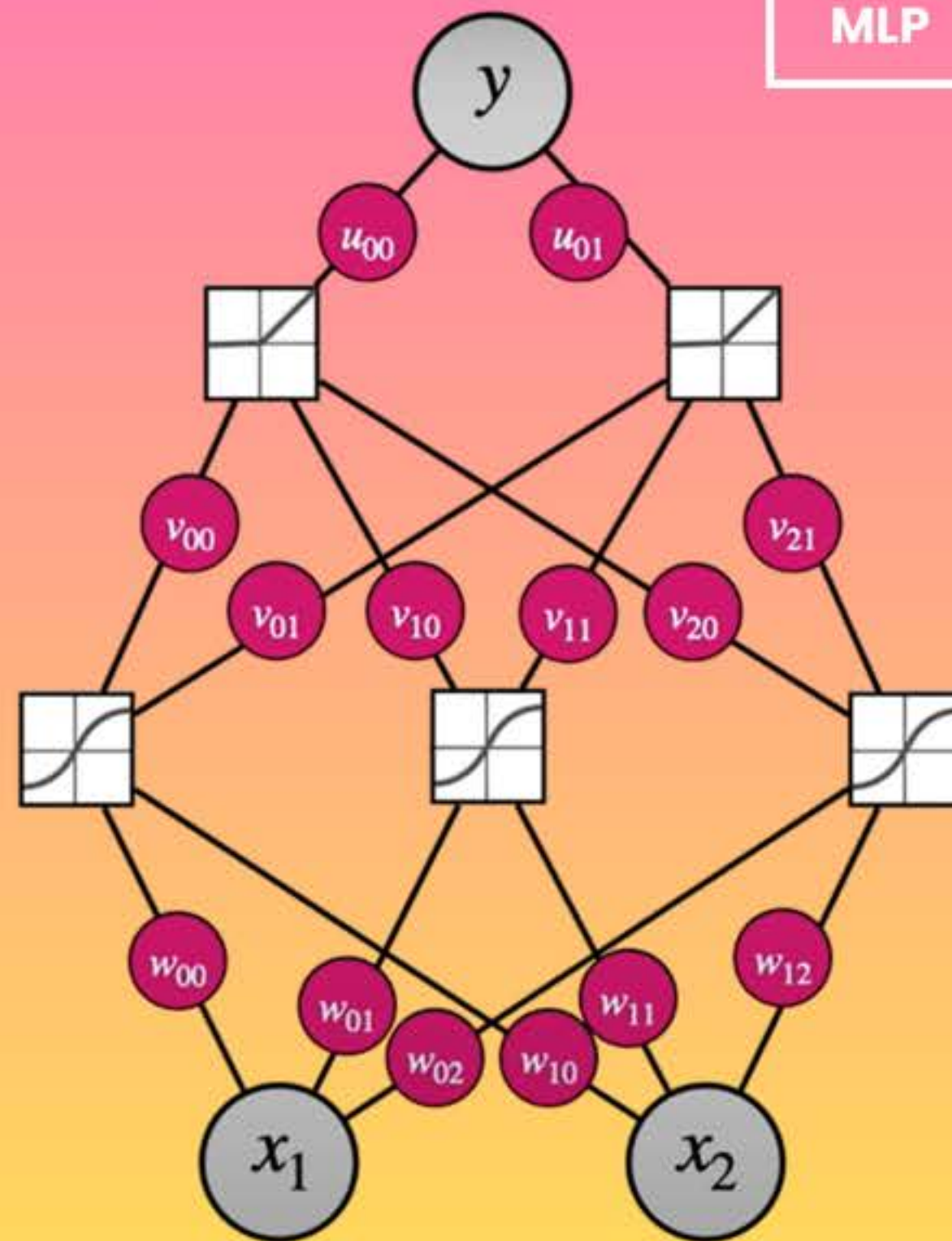


# MLP VS KAN

KAN



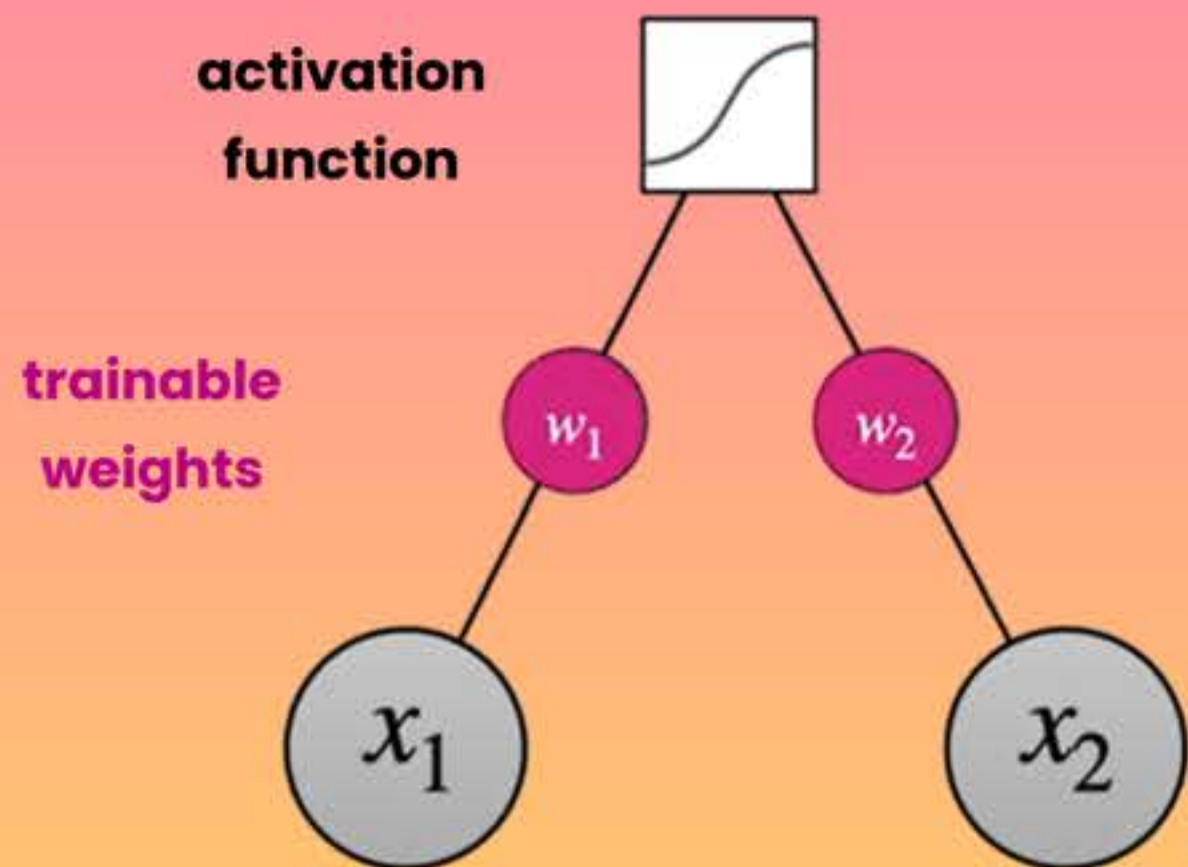
MLP



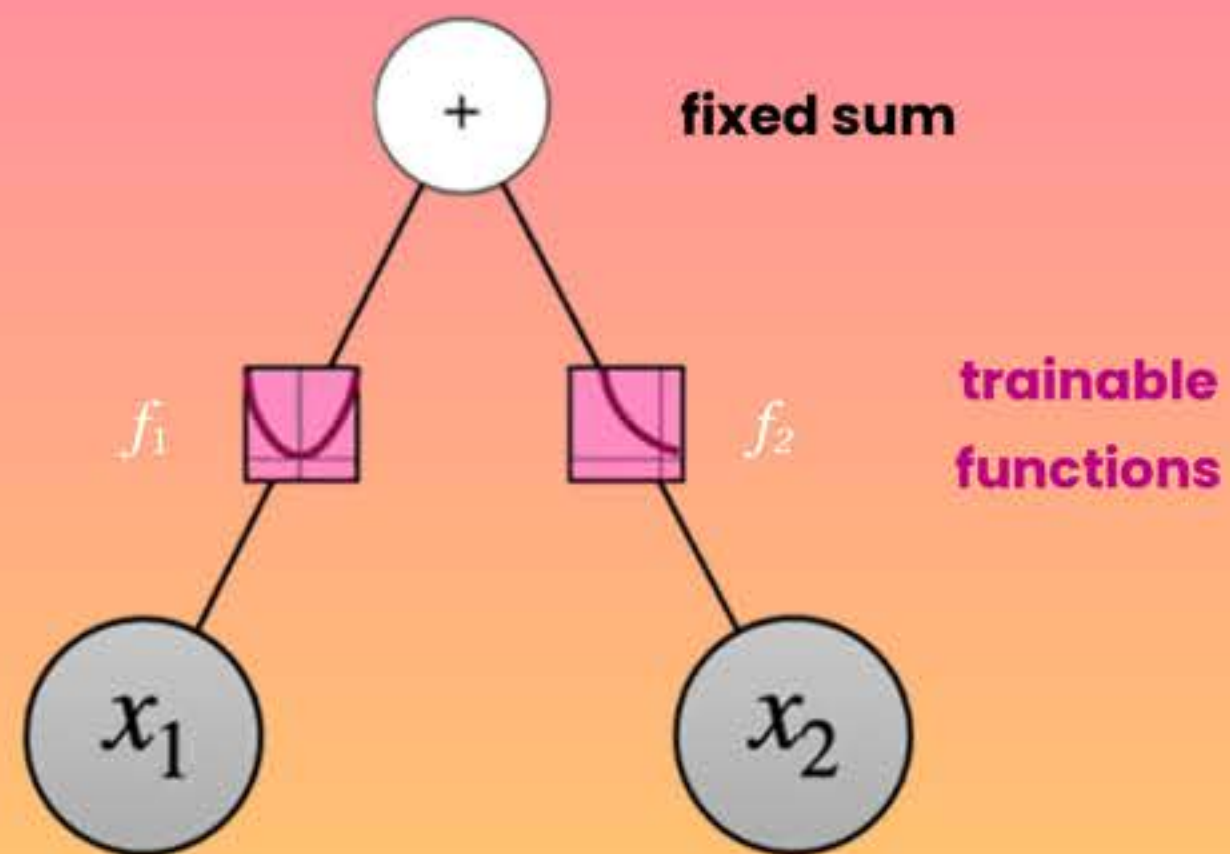
# MLP VS KAN

KAN

MLP



output :  $\sigma(\mathbf{w}_1x_1 + \mathbf{w}_2x_2)$



output :  $\mathbf{f}_1(x_1) + \mathbf{f}_2(x_2)$



# LEARNING SOFT KINEMATICS

## General Idea

The objective is to approximate the mapping from the **end-effector** position in Cartesian space to the actuator space, enabling data-driven inverse models that bypass the need for complex analytical formulations.

## Issue

This is motivated by the **limitations of traditional model-based** methods in capturing the **nonlinear** and **highly deformable behavior** of **soft robots**. We formulate the bending task as a regression problem, where neural networks are trained to estimate the actuation inputs required to achieve a desired tip position of the soft manipulator.





# CASE OF STUDIES







# EFFICIENCY & INTERPRETABILITY



# DATASETS

## Bending Dataset

The dataset used comes from a data collection on a soft robot driven by 9 motors, divided into three functional groups: bending (motors 1-3), twisting CW (4-6) and twisting CCW (7-9). For our experiments we focused exclusively on the bending mode, in which the three motors are controlled to generate a curve of the robot body along a plane.

The collected data includes:

- Cable lengths (actuation) for each motor
- 3D position of markers along the robot body (in particular the last marker, used as a tip representation)
- Two separate trials (t1 and t2), each containing 10,000 samples (1000 points with a factor of 10 smoothing)
- Acquisition rate of 10 Hz

These data provide a realistic and controlled basis for testing inverse (position  $\rightarrow$  actuation) and forward (actuation  $\rightarrow$  position) kinematics models in complex but reproducible deformation scenarios.



# ARCHITECTURES

## KAN Architecture

It has **two spline layers**:

- **Layer 1**: 64 units, 16 knots,, batch-norm, dropout 0.1
- **Layer 2**: 32 units, 12 knots, no batch-norm, no dropout

Reducing the number of hidden units, knots, and dropout in the KAN model would **lower its capacity**, potentially improving generalization but limiting its ability to fit complex trajectories.

## MLP Architecture

It has **two dense layers**:

- **Layer 1**: 64 units, ReLU.
- **Layer 2**: 32 units, ReLU.

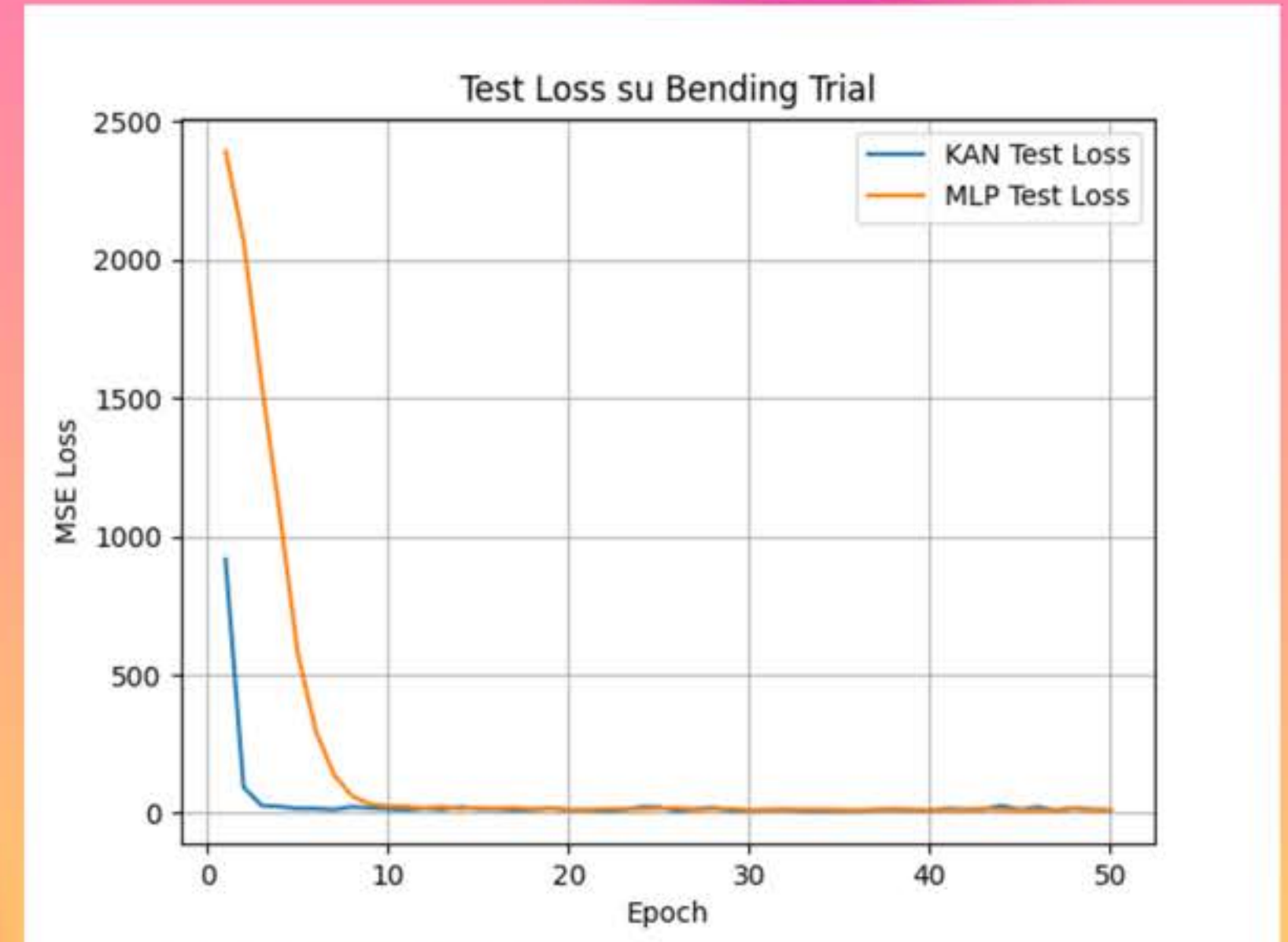
| Component    | KAN                  | MLP                  |
|--------------|----------------------|----------------------|
| Layers       | 2 spline layers      | 2 dense layers       |
| Hidden units | [64, 32]             | [64, 32]             |
| Knots        | [16, 12]             | —                    |
| Activations  | —                    | ReLU                 |
| Dropout      | [0.1, 0.0]           | 0                    |
| Output       | Linear → 3D position | Linear → 3D position |



# CONVERGENCE TIME

We use **50 epochs** for the training.

| Model | Test loss (MSE) | Epochs to converge | Time/epoch | Est. total time |
|-------|-----------------|--------------------|------------|-----------------|
| KAN   | 802             | 6                  | 0.4 s      | ~2.4 s          |
| MLP   | 1.508           | 12                 | 0.3 s      | ~3.6 s          |





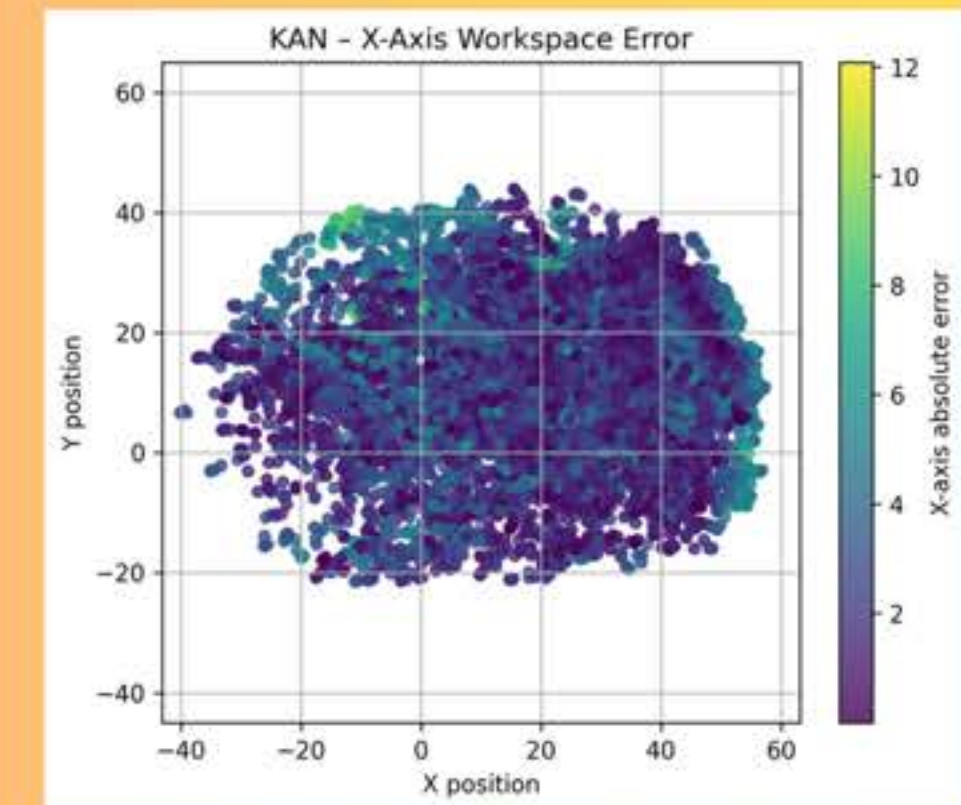
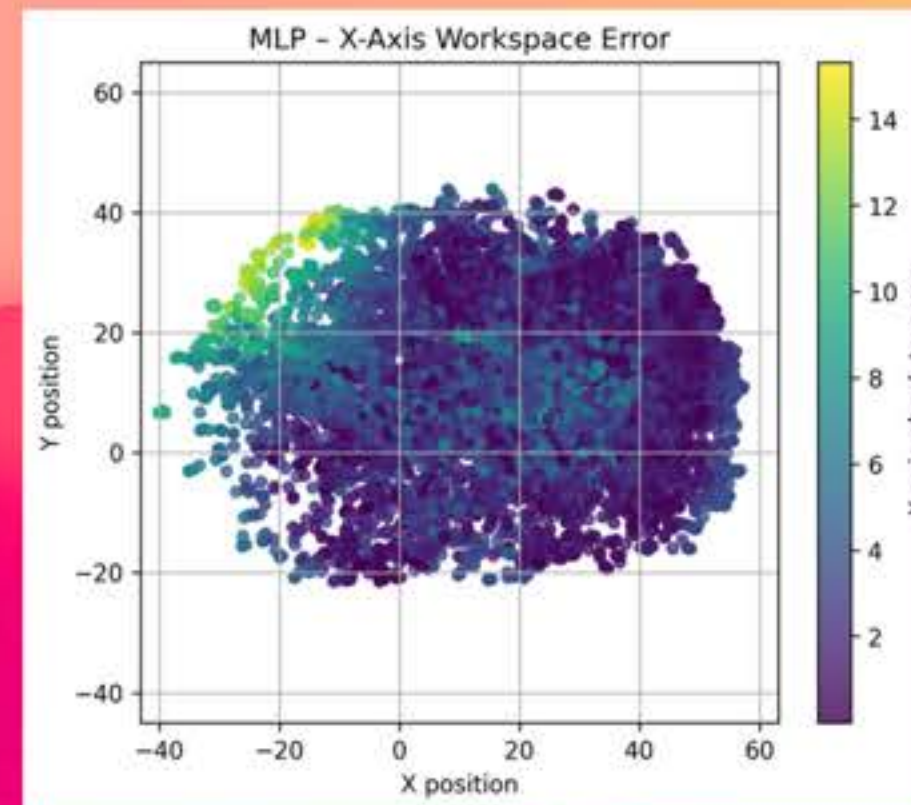
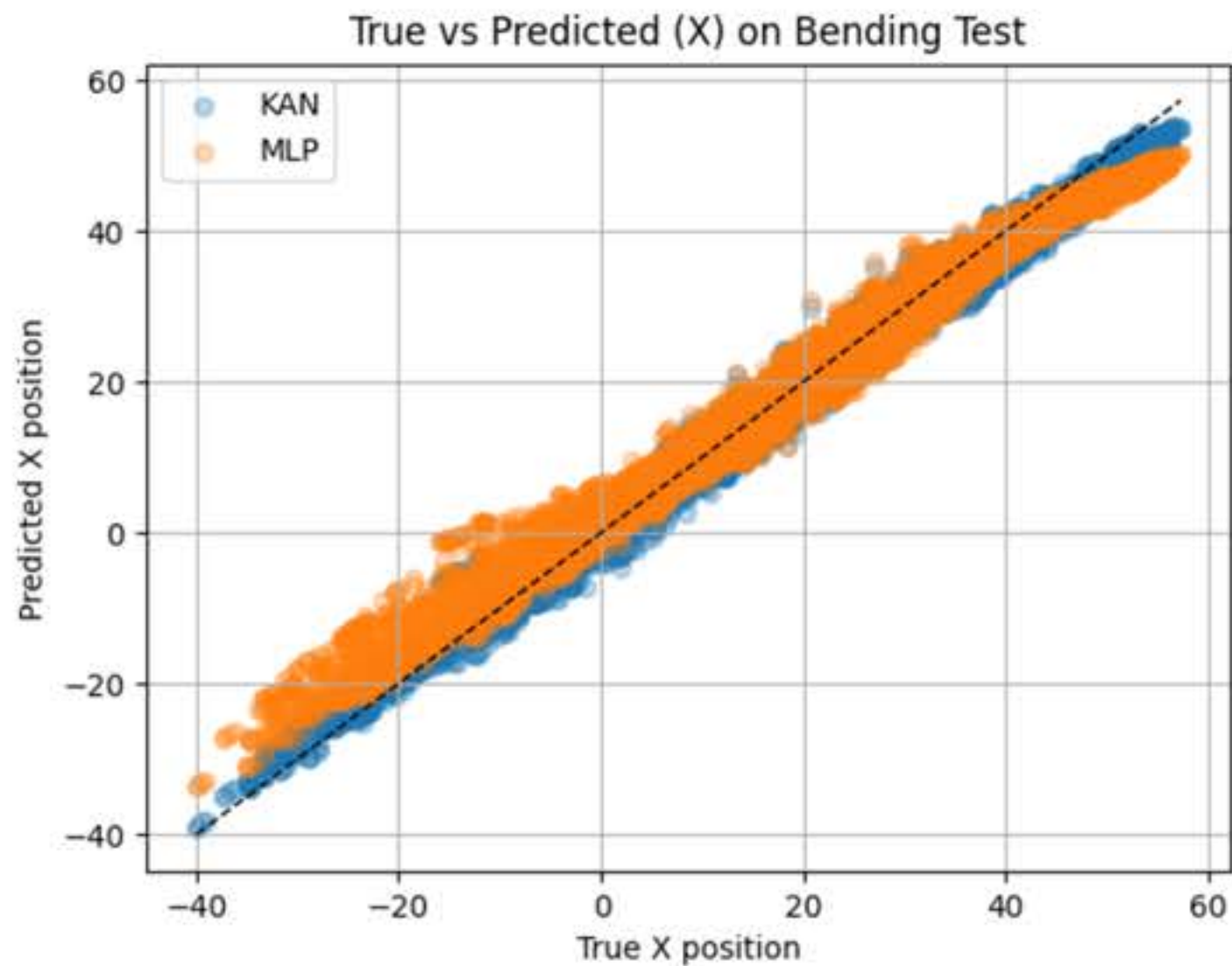
# TEST EVALUATION

KAN → RMSE = 2.4806,  $R^2 = 0.9860$

MLP → RMSE = 3.3276,  $R^2 = 0.9748$

The **KAN** predicts with very high accuracy, aligning the points *almost perfectly* on the ideal diagonal.

The **MLP**, on the other hand, has a *higher dispersion* and bias, especially when the true position is far from the center.

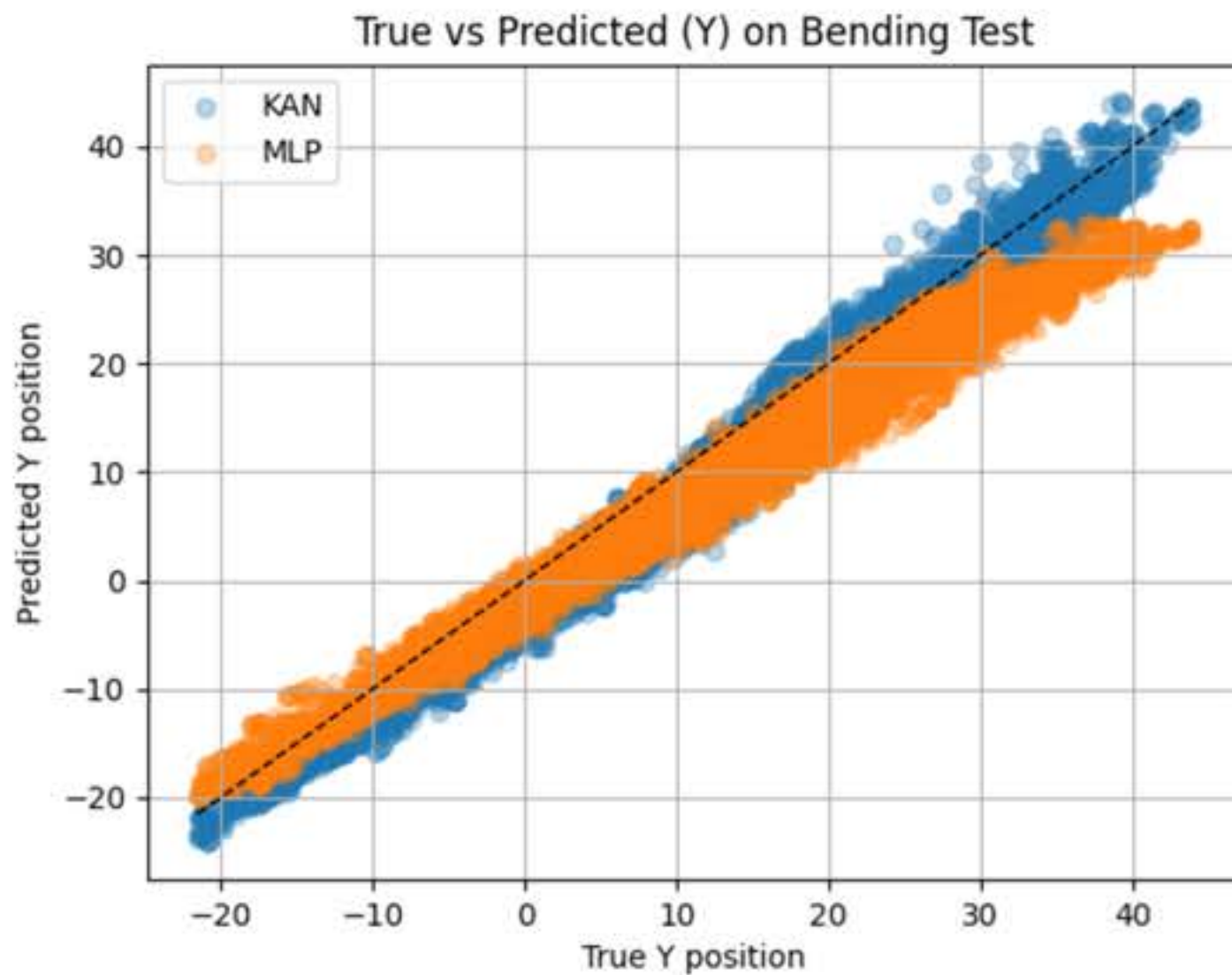




# TEST EVALUATION

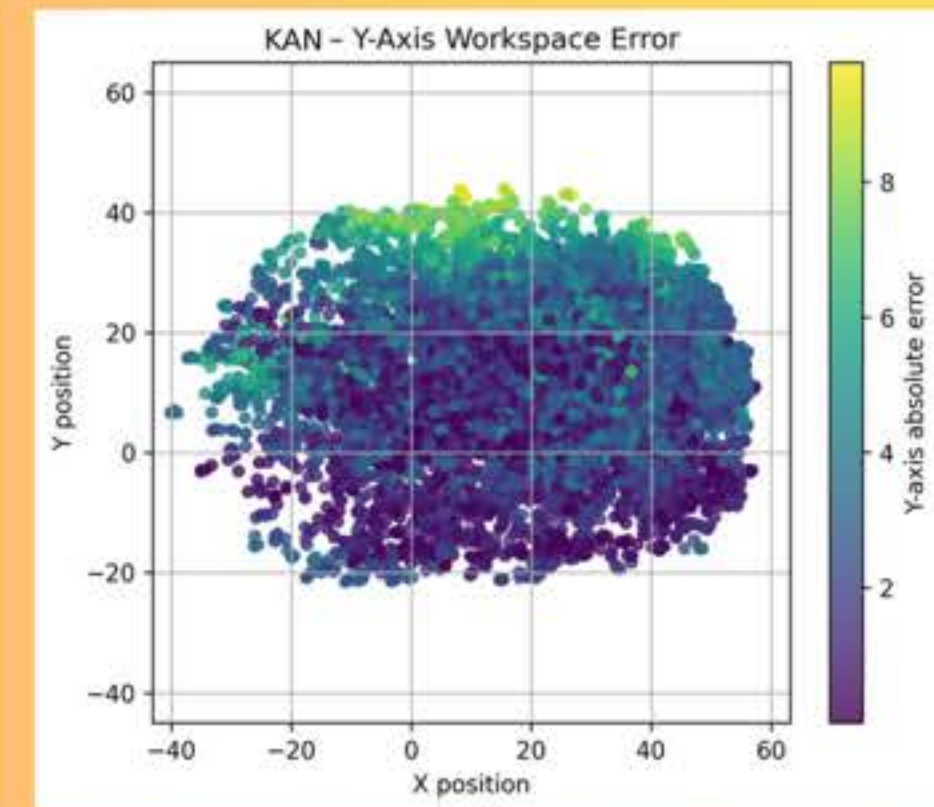
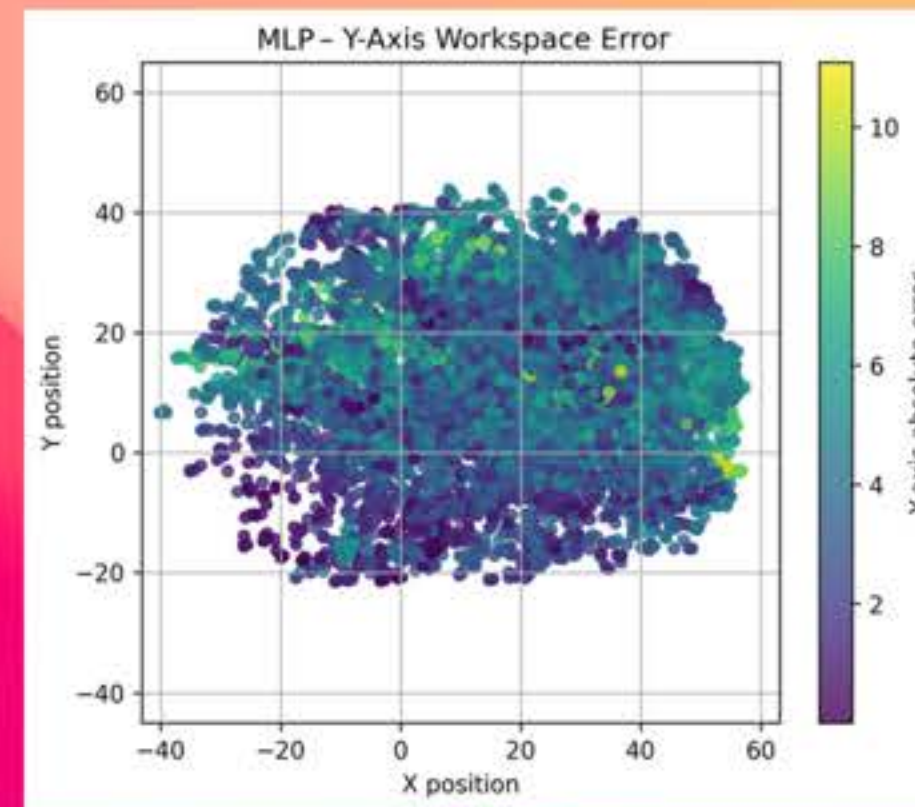
KAN → RMSE = 3.1429,  $R^2 = 0.9430$

MLP → RMSE = 4.1780,  $R^2 = 0.8993$



The *superiority* of **KAN** clearly emerges: its predictions are more *densely distributed* along the diagonal.

The **MLP** tends to *underestimate* the position with a systematic bias.





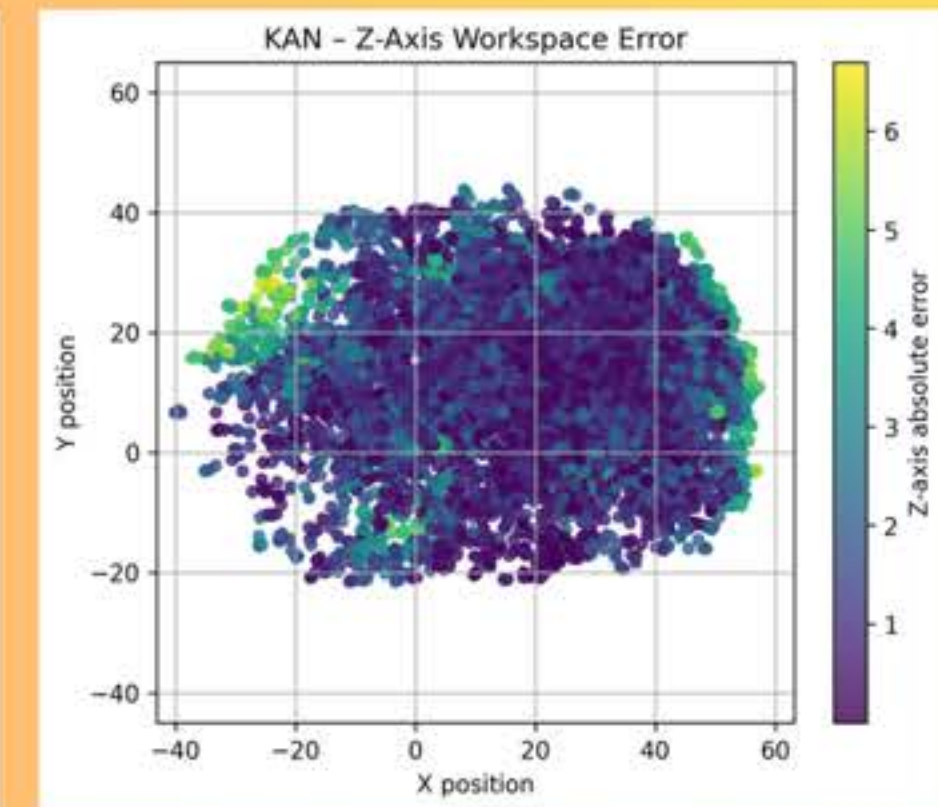
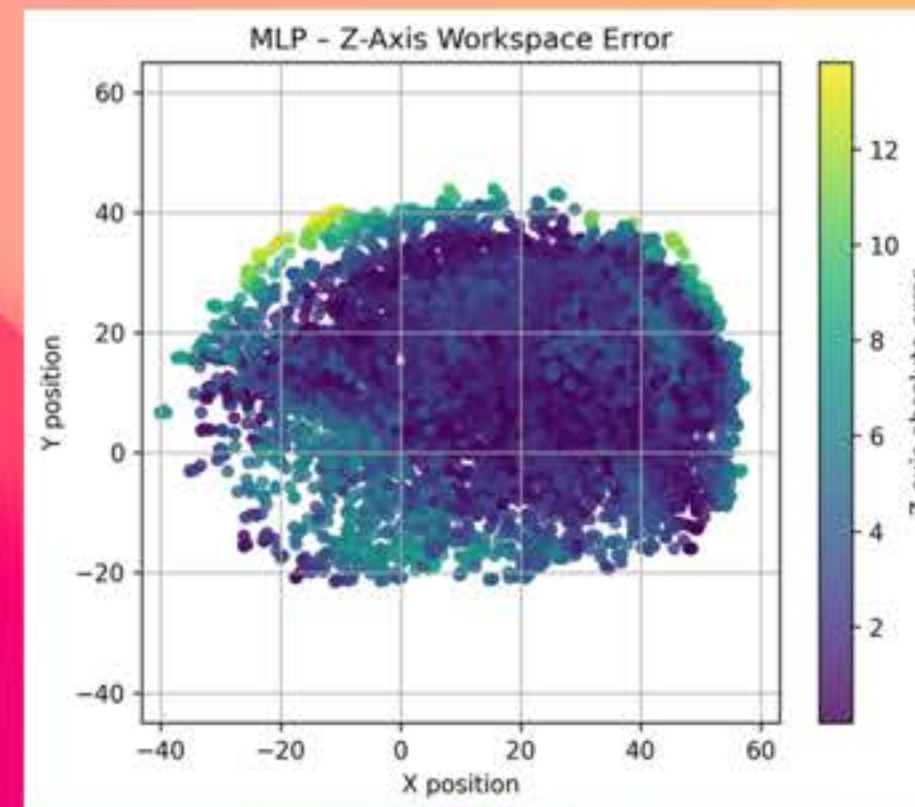
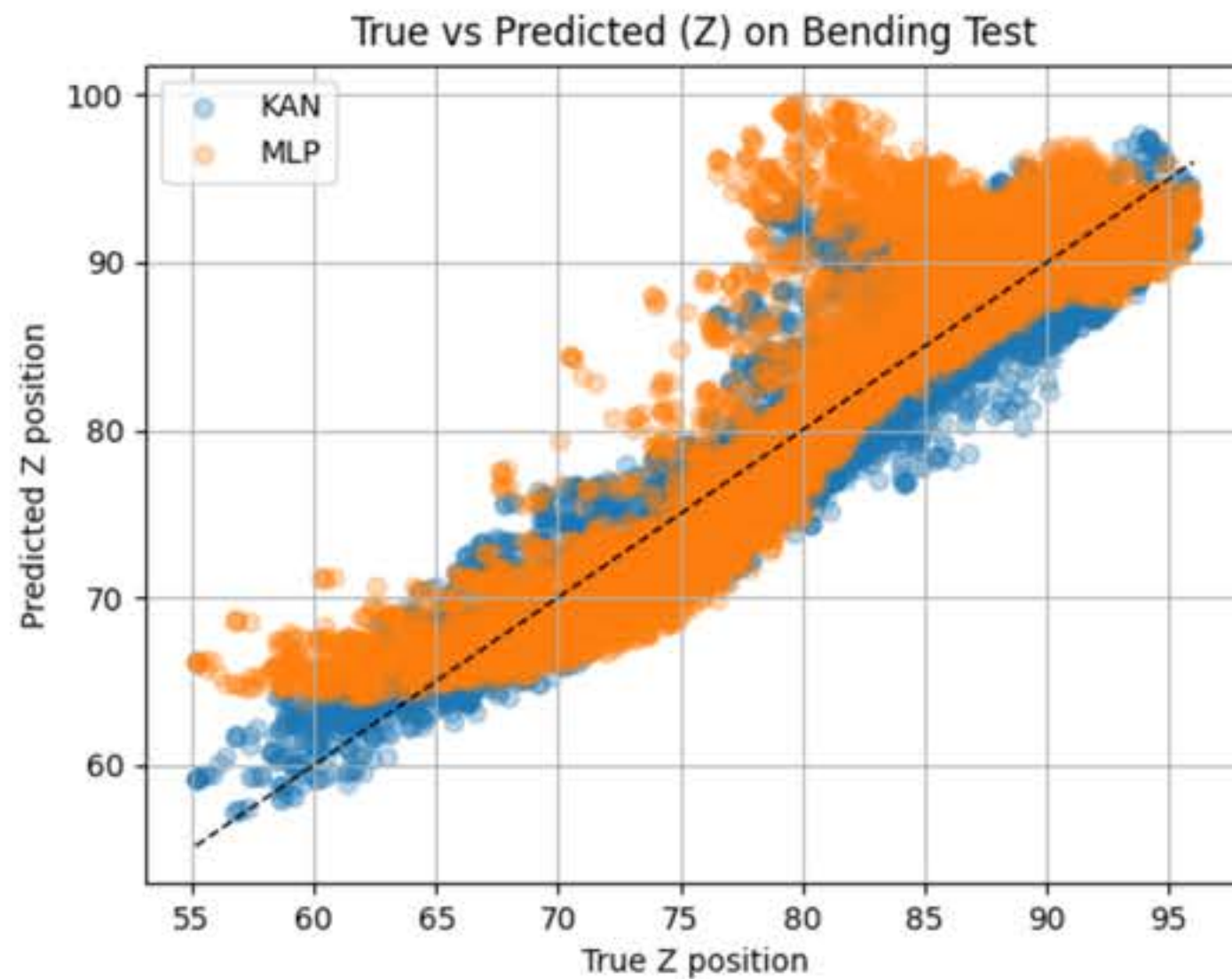
# TEST EVALUATION

KAN → RMSE = 2.8321,  $R^2$  = 0.8843

MLP → RMSE = 4.0897,  $R^2$  = 0.7587

The **KAN** shows a more stable and precise behavior.

The **MLP** tends to *saturate* towards the top, losing generalization capacity for high values of Z.





# INTERPRETABILITY

The analysis of **splines** learned from the layers of the KAN (Kernel-based Activation Network) network allows to understand the internal behavior of the model in the nonlinear transformation of the inputs. Each spline represents the **activation function learned** from a neuron, defined on an input range, and provides an interpretable representation of the contribution of each feature.

BY ANALYZING THESE BEHAVIORS, IT IS POSSIBLE TO OBTAIN A QUALITATIVE READING OF THE GENERALIZATION CAPACITY AND COMPLEXITY OF THE MODEL.



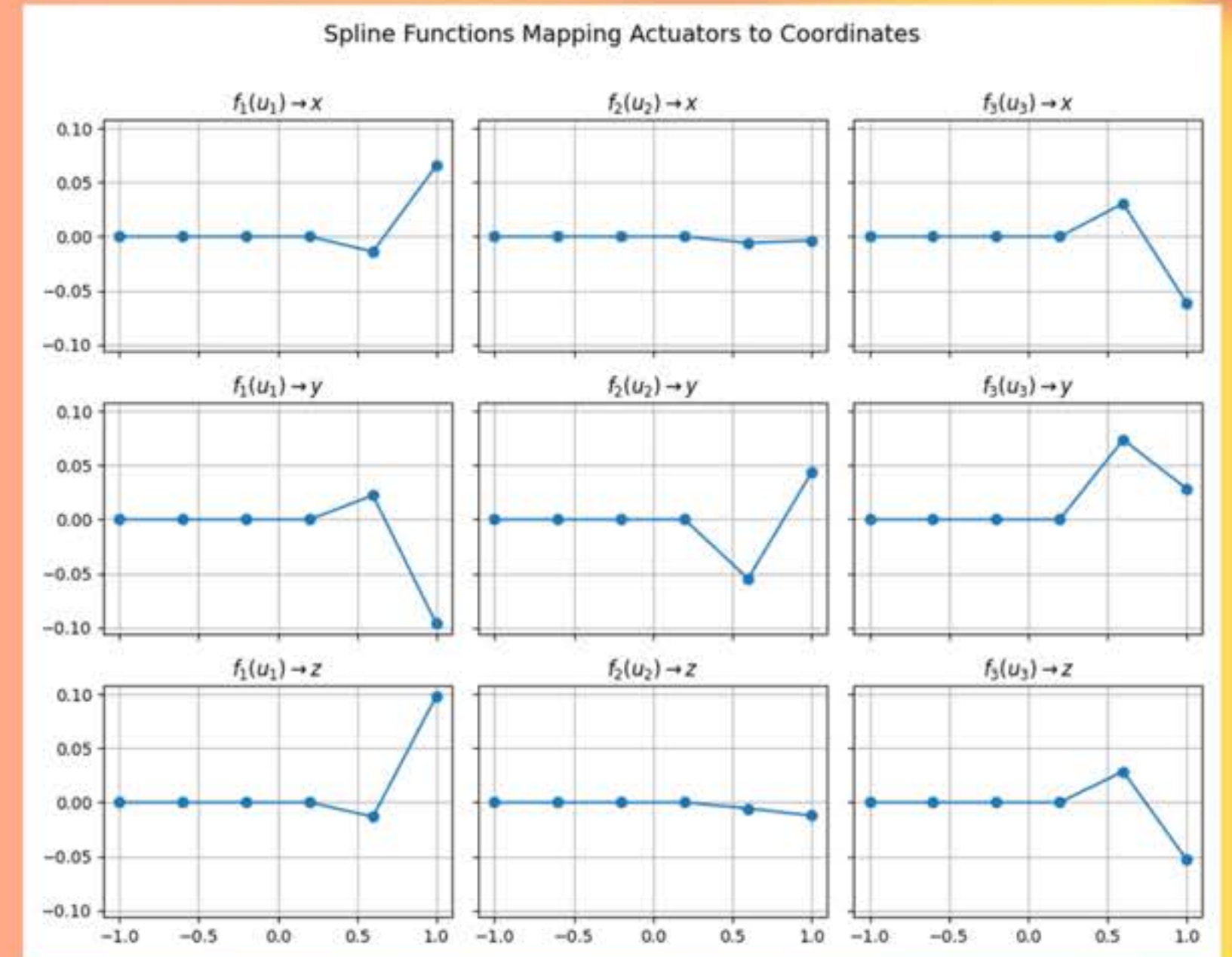


# ANALYTICAL INTERPRETATION OF A MINIMAL KAN

To better understand how KAN operates, we reduced the network to its simplest form: a single hidden layer with three inputs (the actuators  $u_1, u_2, u_3$ ) and three outputs (the spatial coordinates  $x, y, z$ ). Each hidden unit applies a piecewise linear spline transformation  $f_j(u_j)$ , which captures non-linearities while preserving interpretability. The output layer then performs a simple weighted sum, allowing the model to remain analytically tractable.

$$\begin{aligned}x &= w_{1x} \cdot f_1(u_1) + w_{2x} \cdot f_2(u_2) + w_{3x} \cdot f_3(u_3) + b_x \\y &= w_{1y} \cdot f_1(u_1) + w_{2y} \cdot f_2(u_2) + w_{3y} \cdot f_3(u_3) + b_y \\z &= w_{1z} \cdot f_1(u_1) + w_{2z} \cdot f_2(u_2) + w_{3z} \cdot f_3(u_3) + b_z\end{aligned}$$

This form shows that the network learns to map each actuator to each coordinate via an interpretable affine composition.







# CONTINUAL LEARNING TEST



# GENERAL IDEA

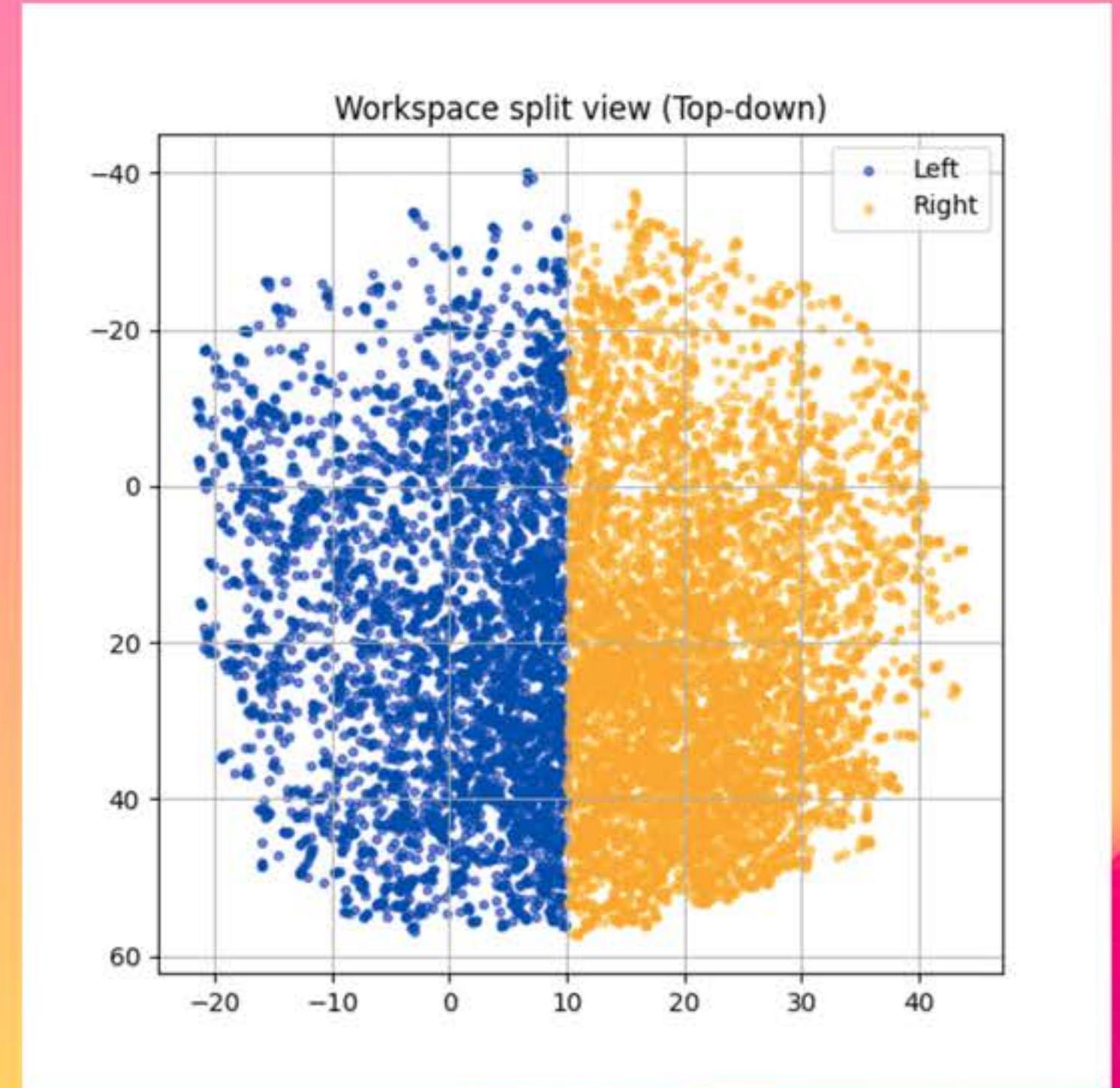
To analyze spatial generalization, we divided the dataset into two regions based on the Y-axis position of the robot's tip:

- **Left workspace:** samples where the tip lies in the negative Y semiplane.
- **Right workspace:** samples where the tip lies in the positive Y semiplane.

We then applied a two-phase continual learning protocol:

1. (**Ms** → **Ds**): Train both KAN and MLP models only on the **left semiplane**. This serves as the source domain, capturing initial motor-space mappings.
2. (**Md** → **Ds/Dd**): Fine-tune the **same models** on the **right semiplane**. We then evaluate performance on:
  3. **MdDs**: old domain (left), to measure forgetting.
  4. **MdDd**: new domain (right), to assess generalization.

This setup allows us to quantify catastrophic forgetting, forward transfer, and compare how KAN and MLP adapt across disjoint motor regions.

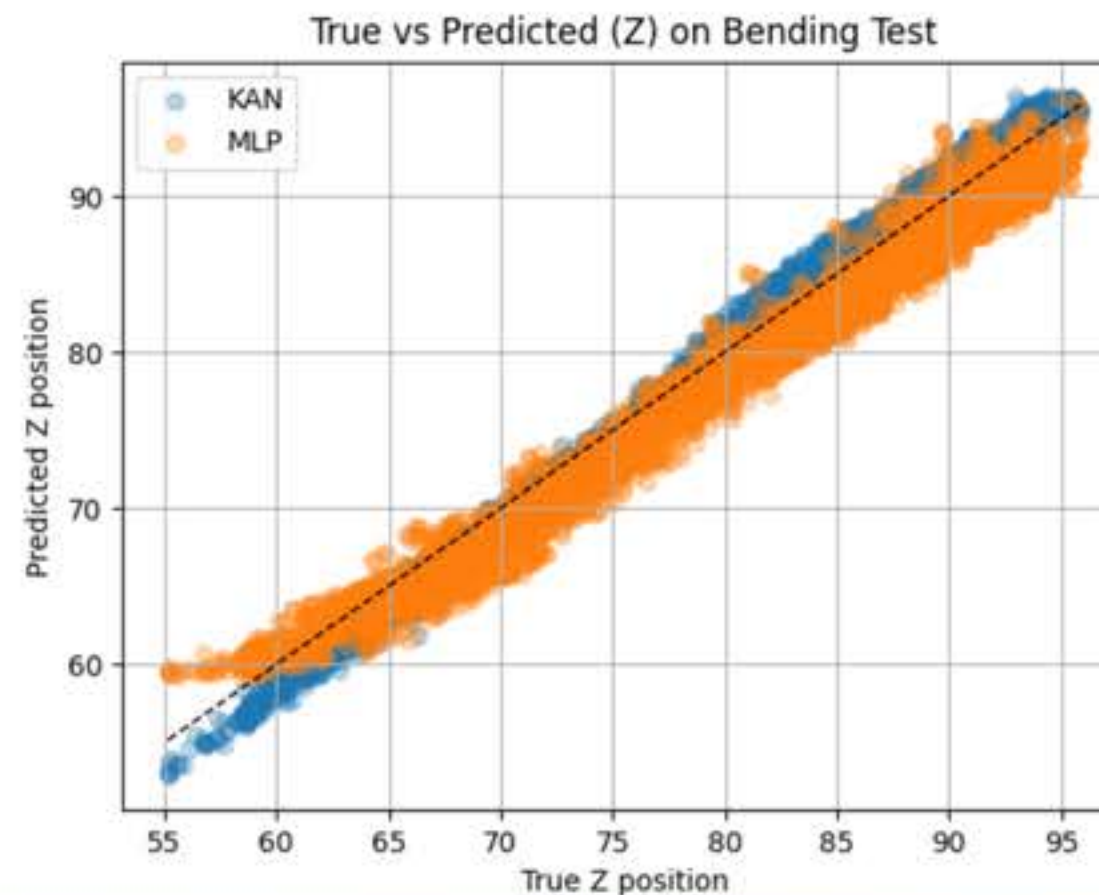
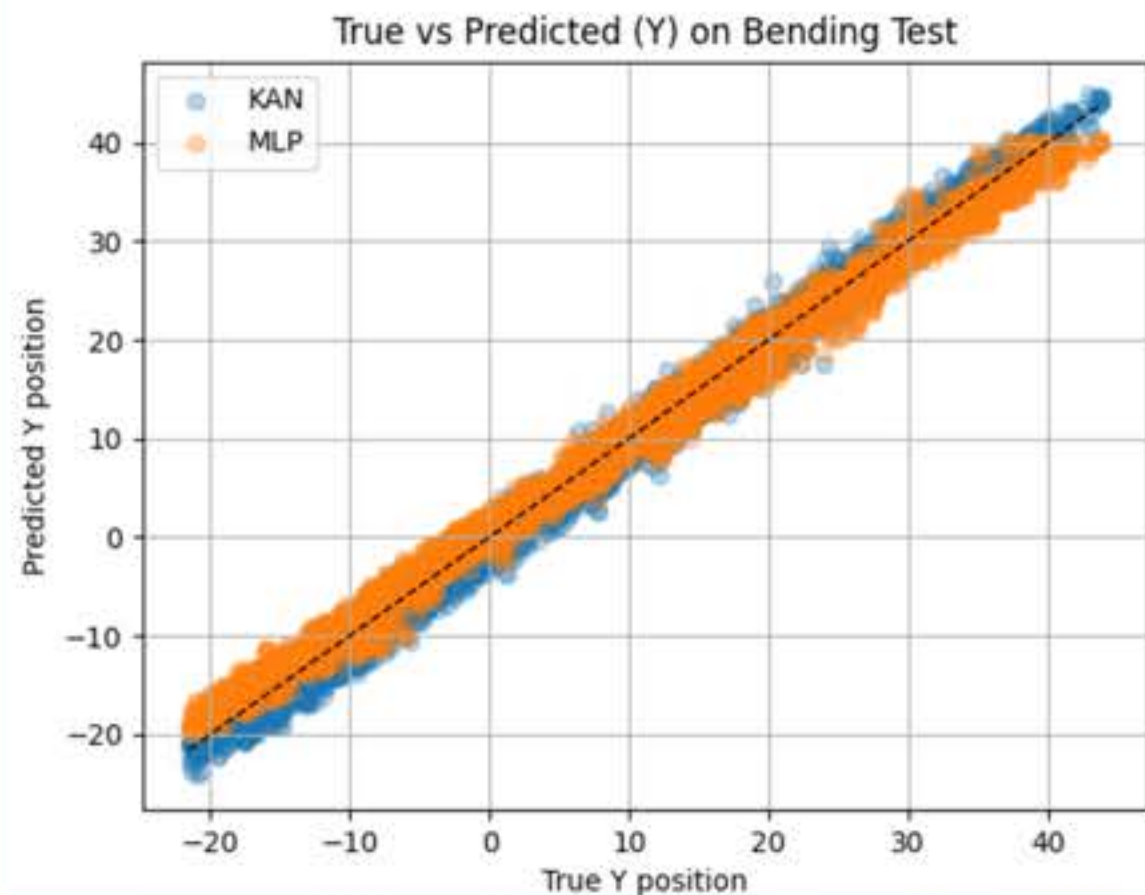
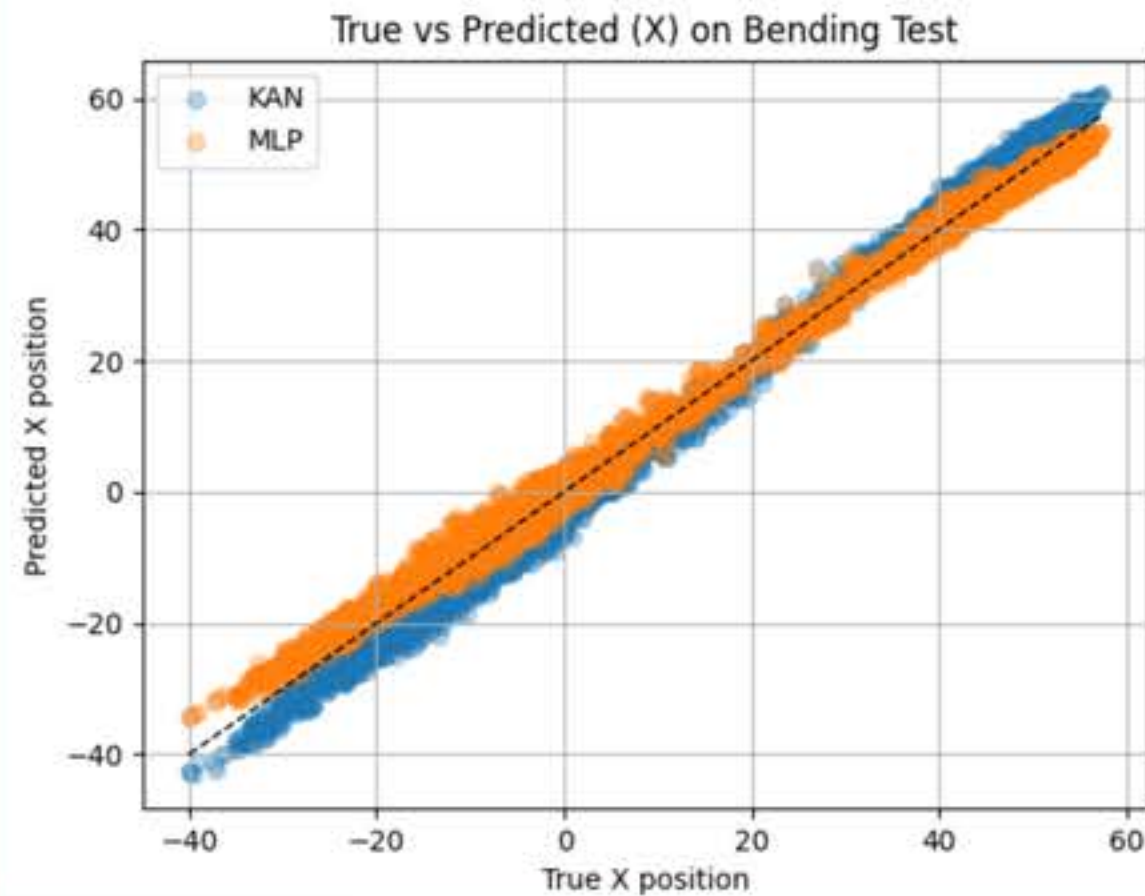




# FIRST LOOK

**(Ms, Ds)**

The two models works very well and they are quite similar. Overall, both models closely follow the ground truth, with KAN having a slight edge in precision.

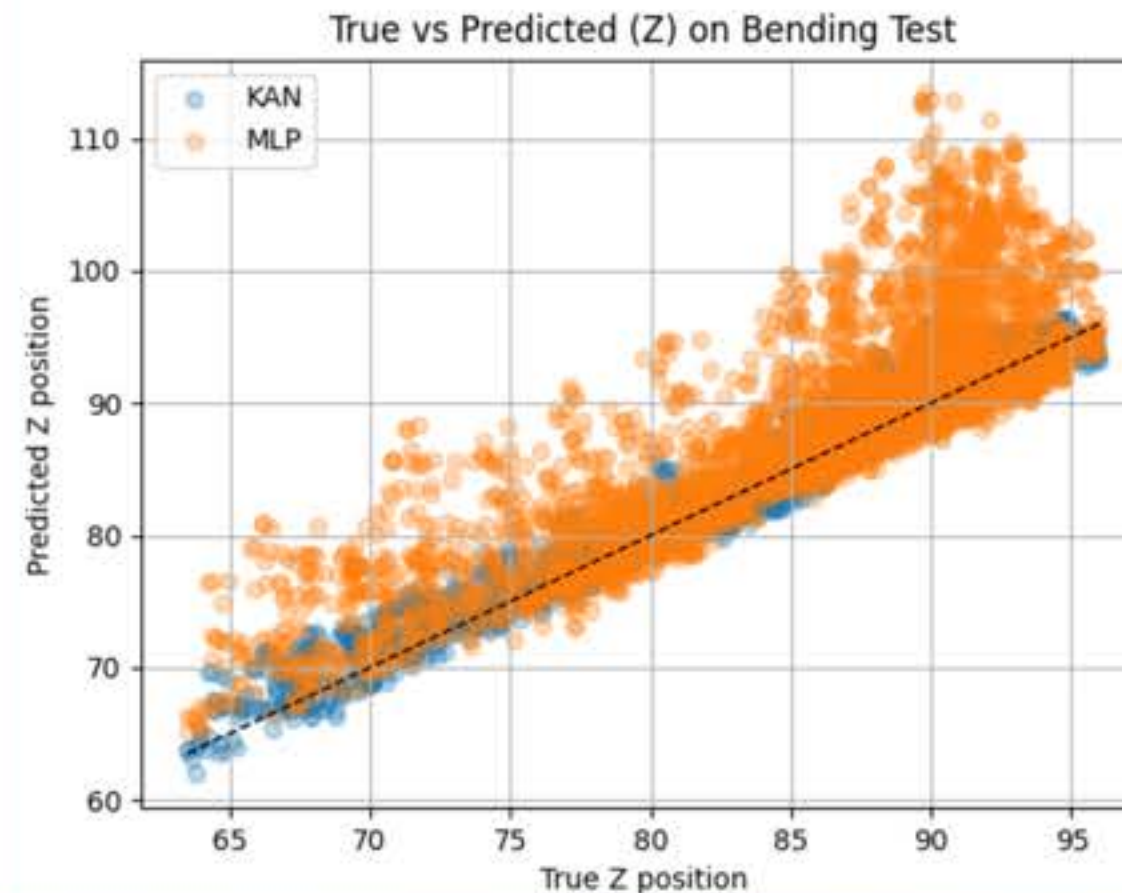
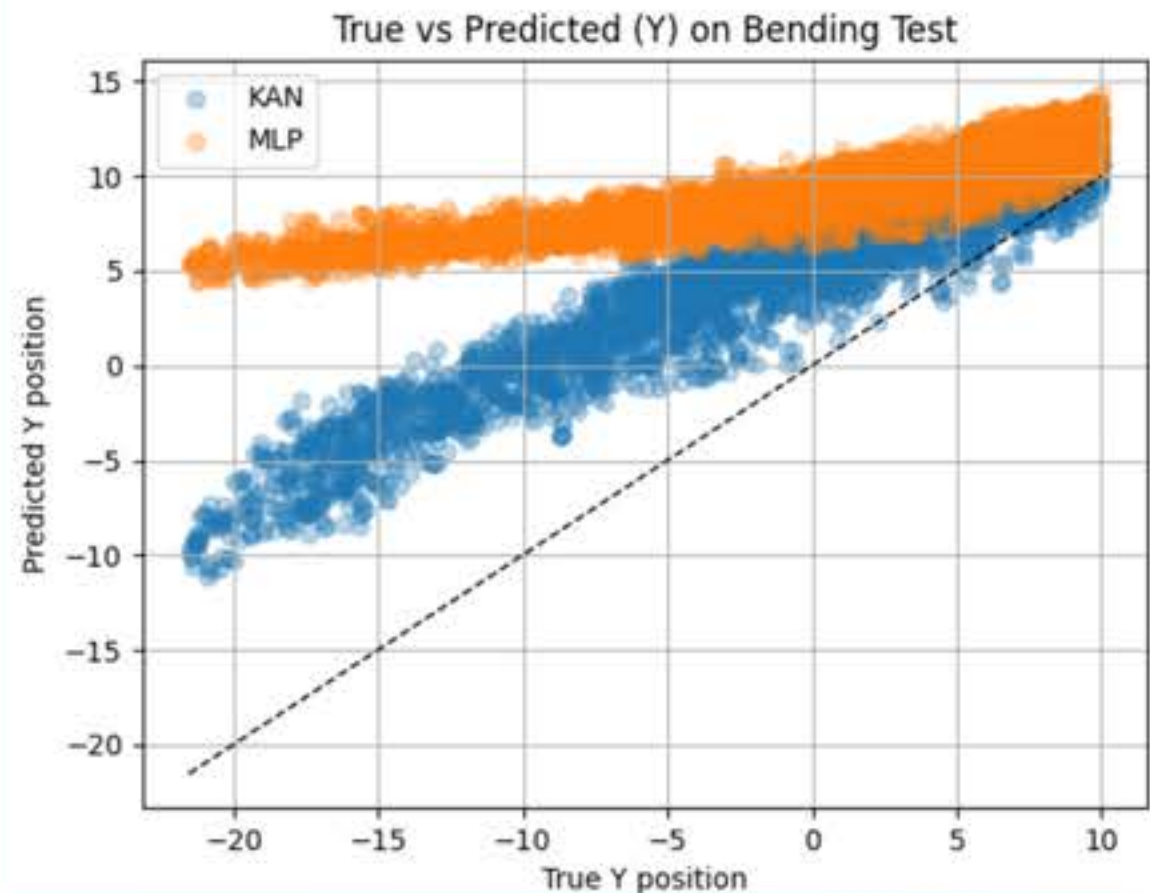
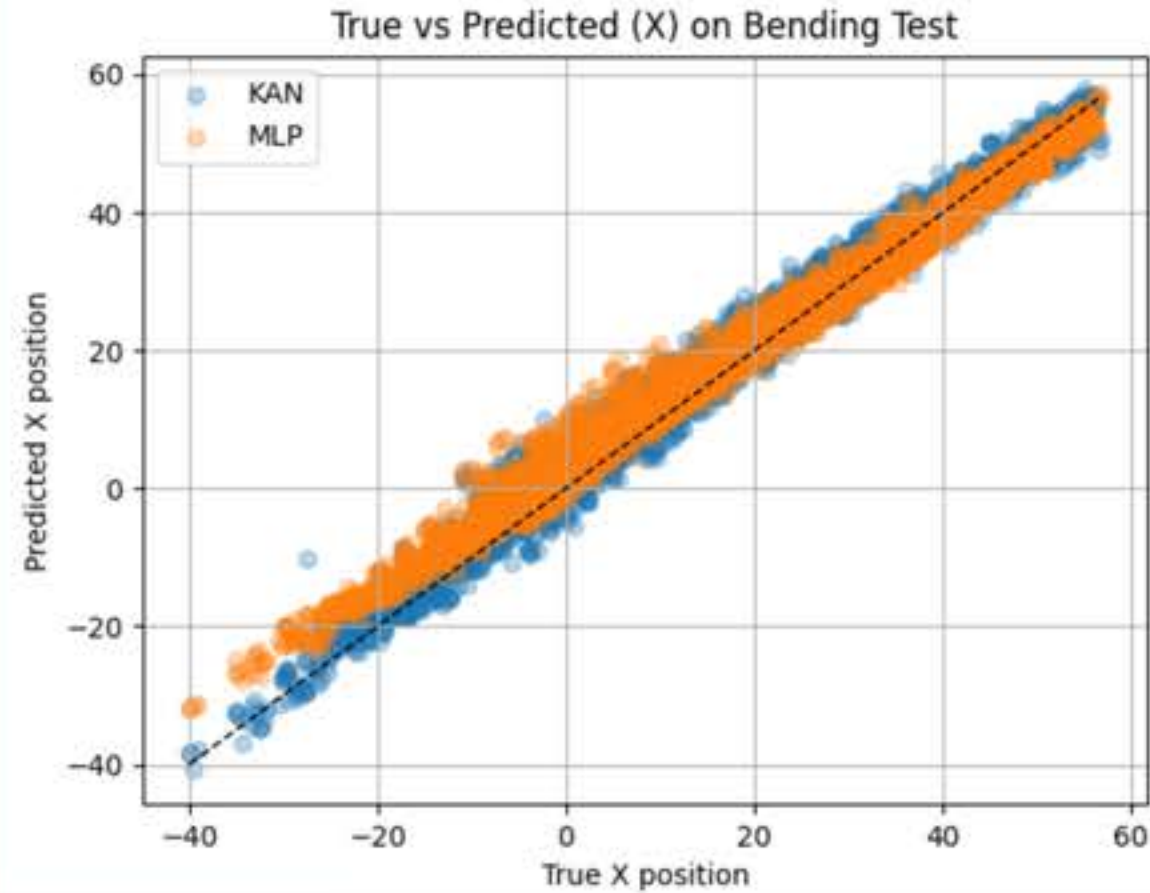




# FIRST LOOK

**(Ms, Ds)**

Here we can see the core of the case of study: the superiority of KAN. The models struggle to generalize the Y axis deformation patterns from the right to the left side, possibly due to asymmetric dynamics or insufficient representation in the training data. The issue is more pronounced in MLP

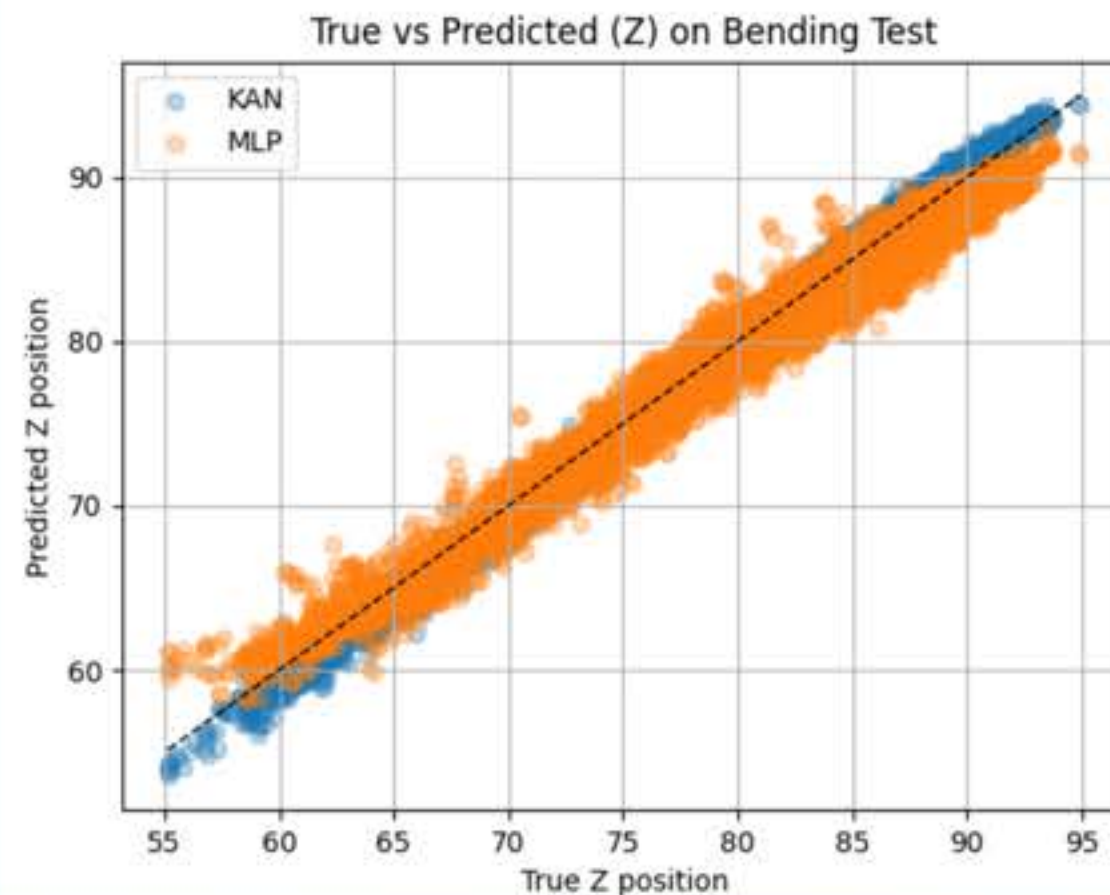
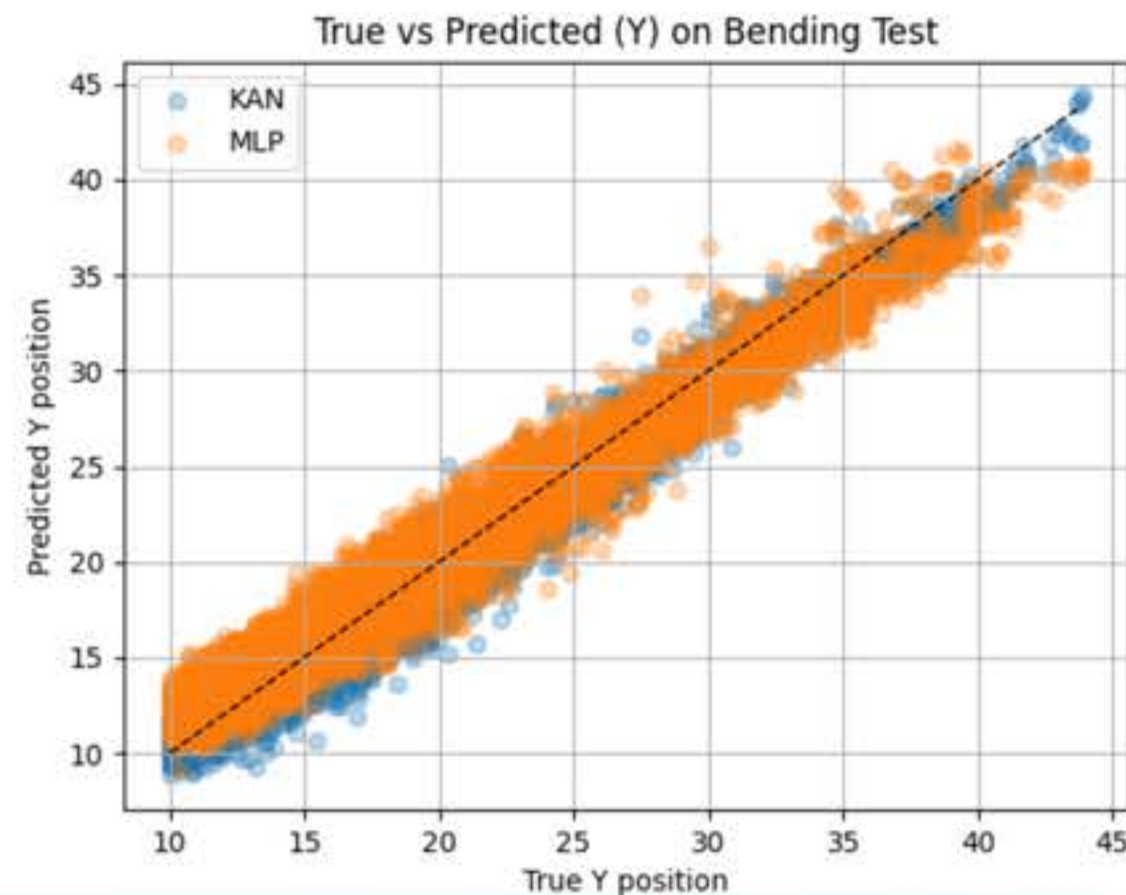
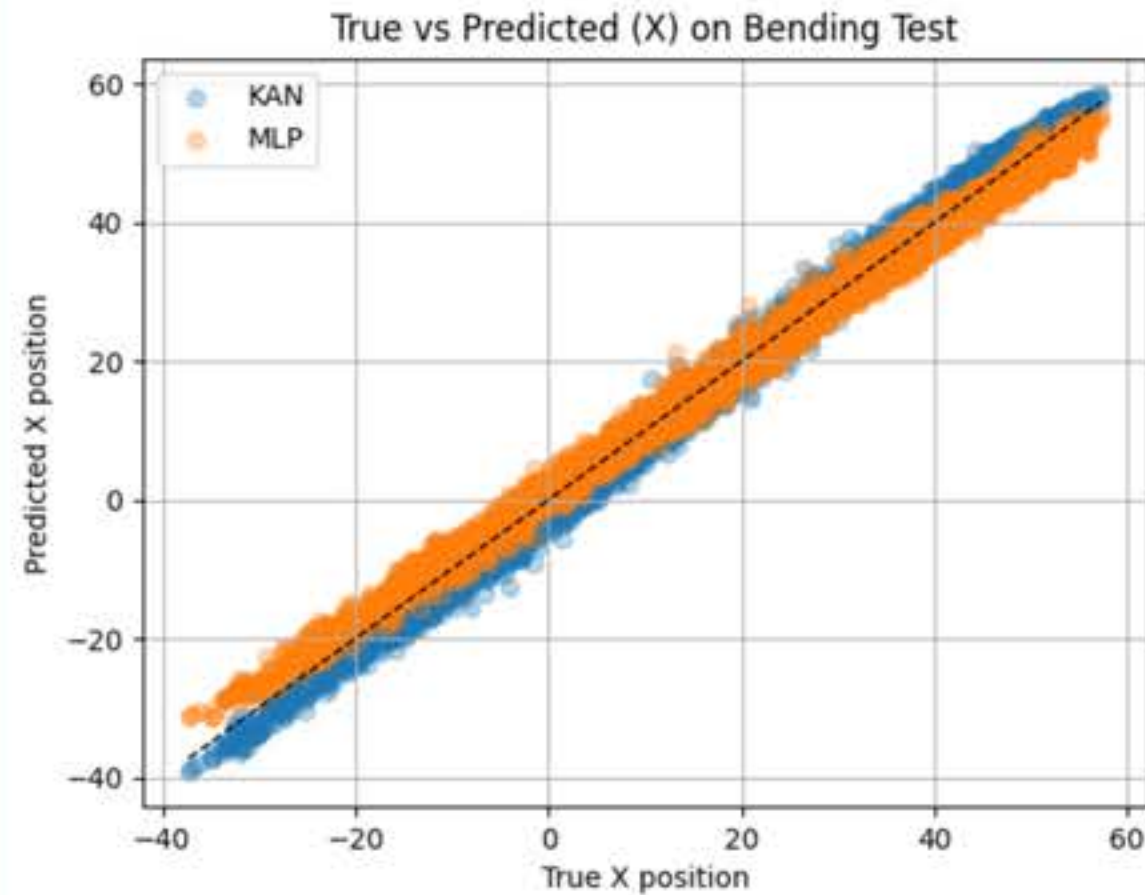




# FIRST LOOK

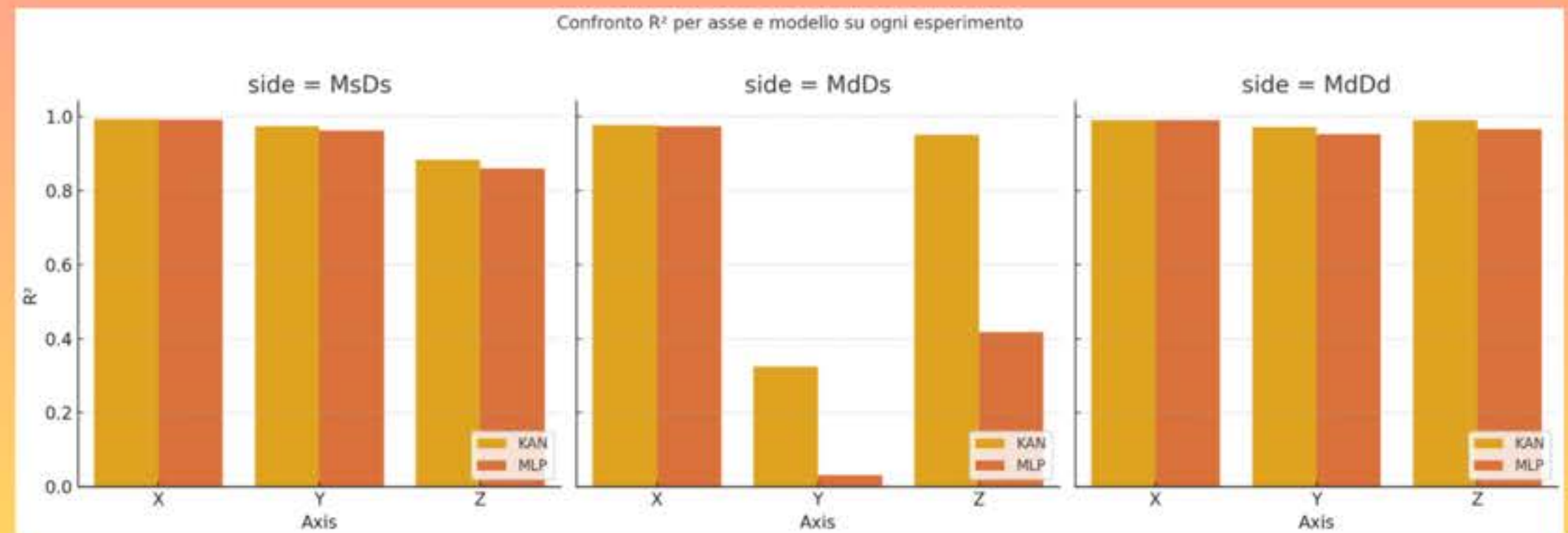
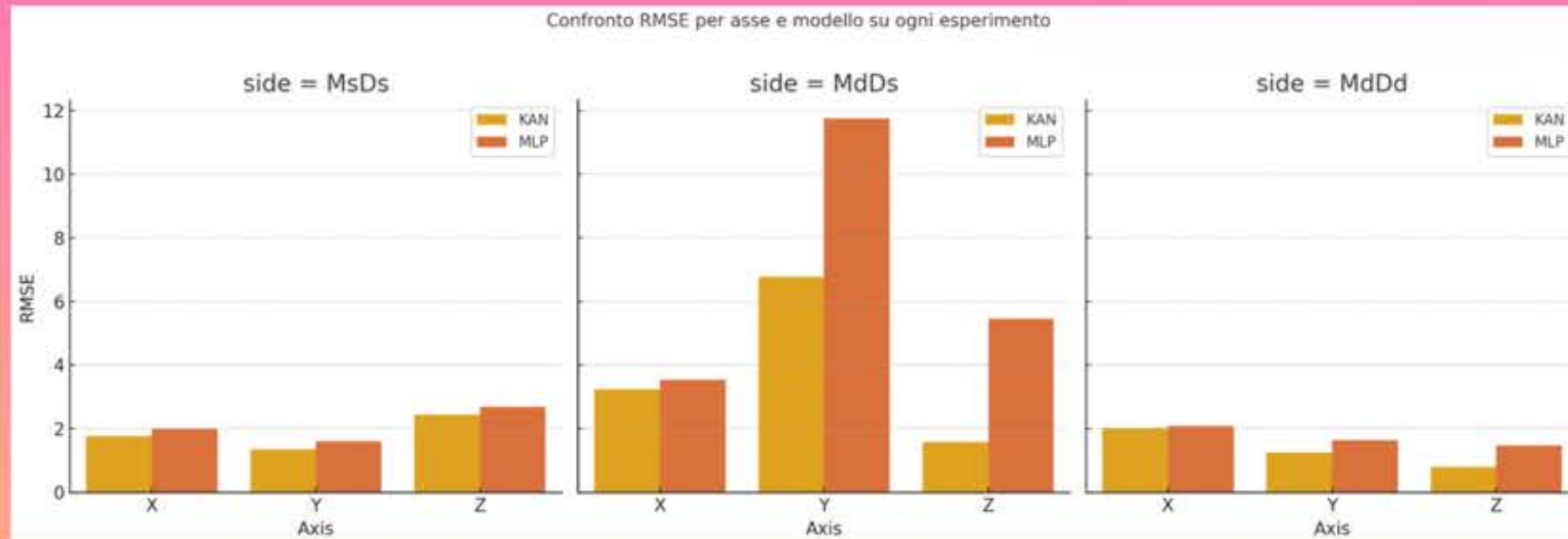
**(Md, Dd)**

Both KAN and MLP models show strong predictive performance across all axes in the bending test. KAN produces slightly more consistent results, especially in the Y and Z directions, while MLP remains accurate but with slightly higher variance.



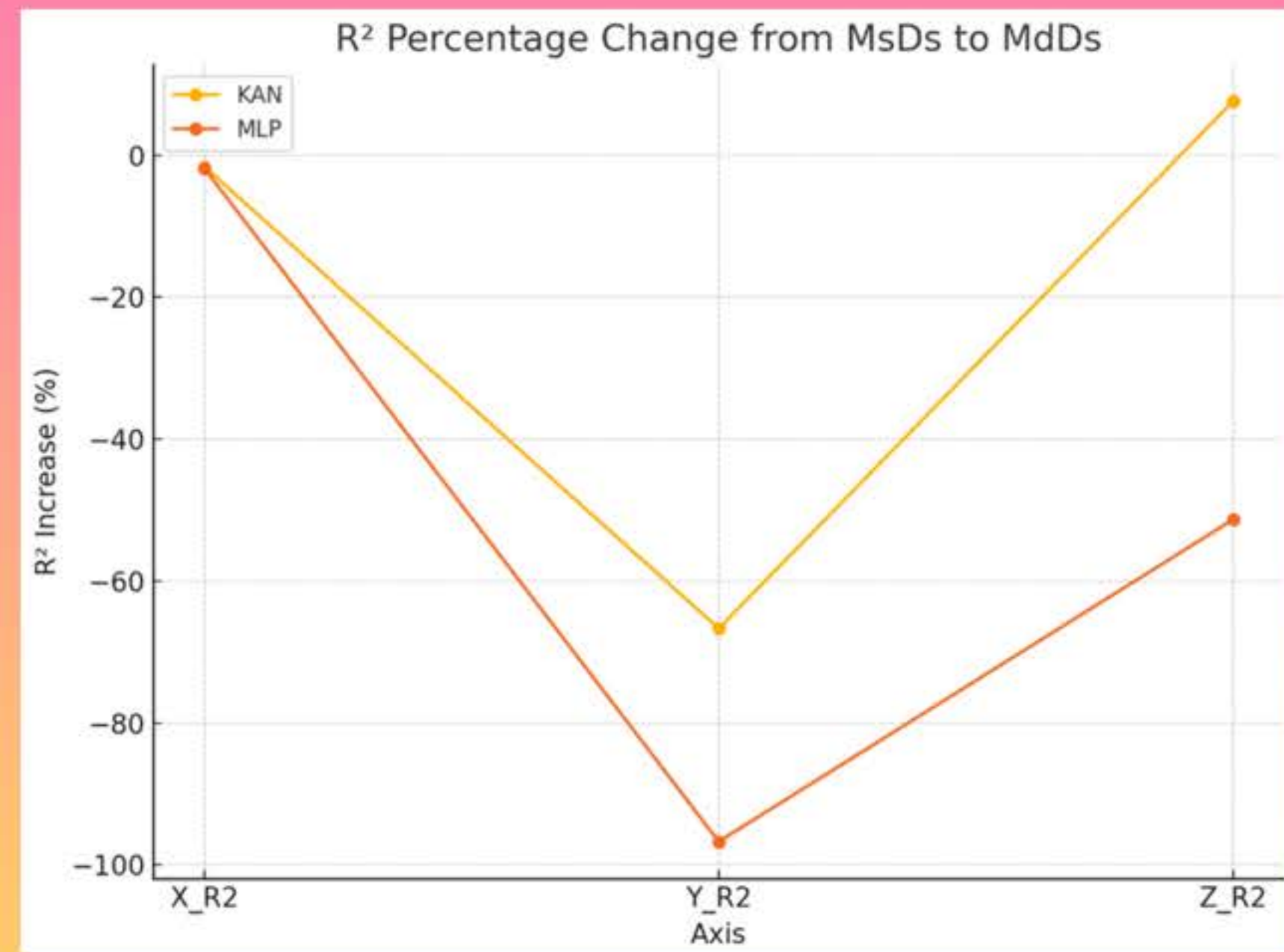
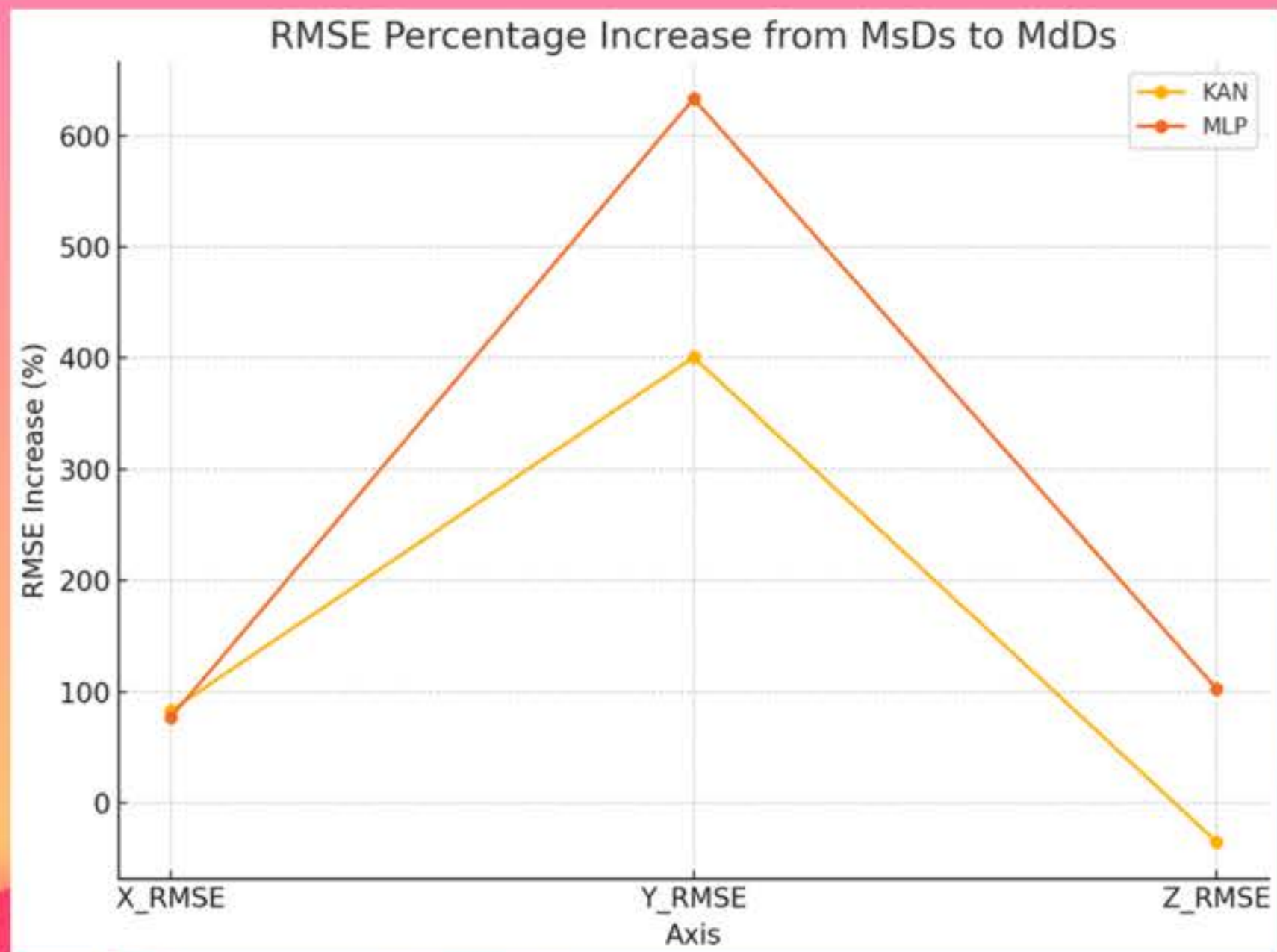


# RESULTS





# RESULTS



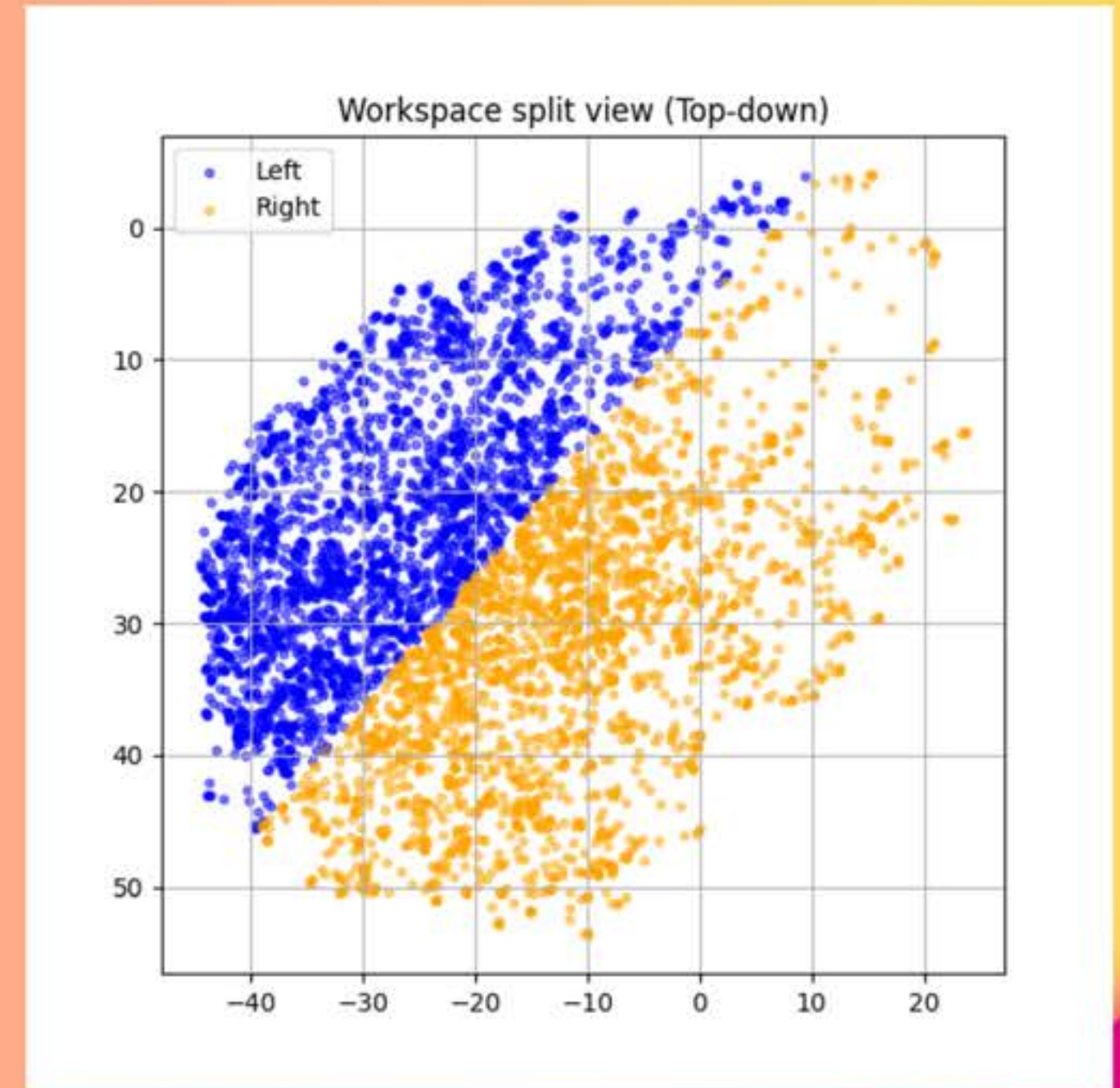


# TWISTING CW

**Twisting Clockwise (CW)** refers to a deformation mode in which the soft manipulator undergoes a rotational motion around its central axis in a clockwise direction when viewed from the base to the tip.

This motion is actuated primarily by motors 4, 5, and 6, as indicated in the dataset description. During CW twisting, the structure experiences torsional forces that cause the tip to rotate while maintaining relatively constant axial displacement.

This deformation plays a key role in tasks requiring orientation control or rotational manipulation. To effectively analyze or model this motion, it is often useful to split the dataset into two semispaces (e.g., along a diagonal axis in the X-Y plane), since the twist introduces symmetrical behavior not aligned with standard Cartesian axes.

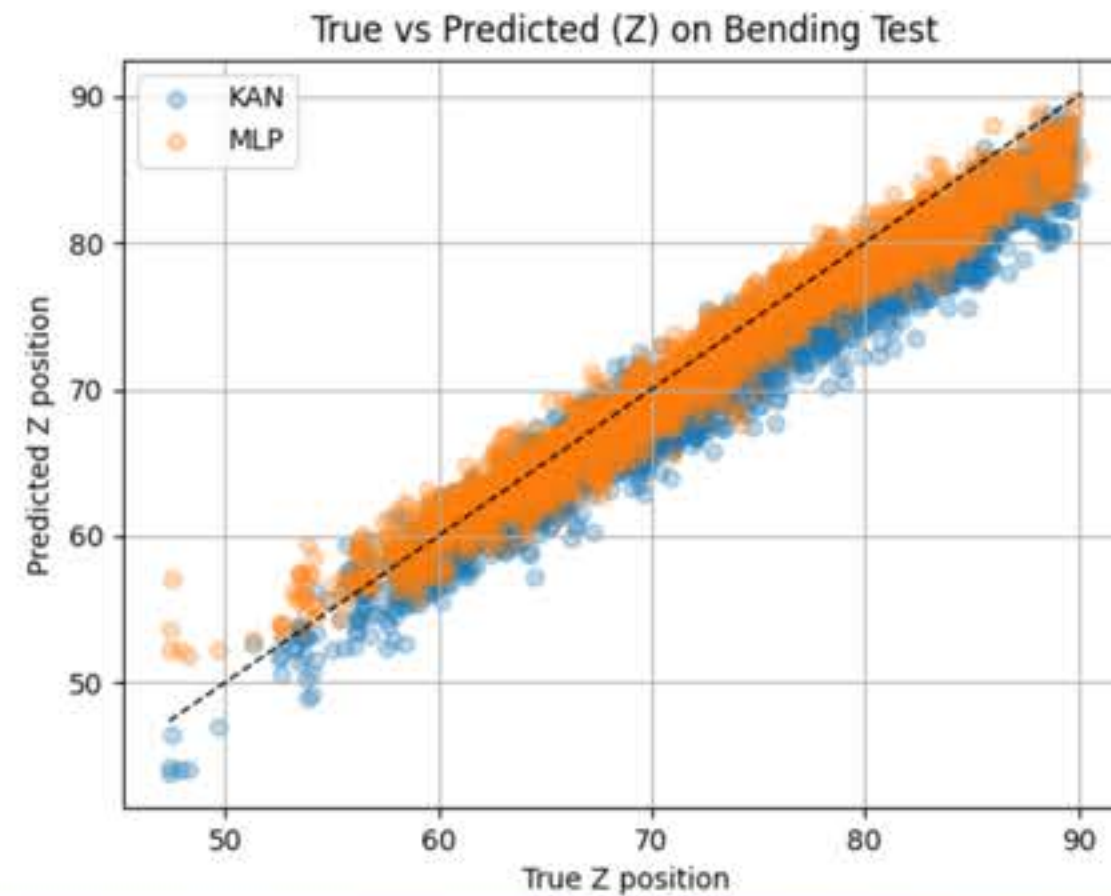
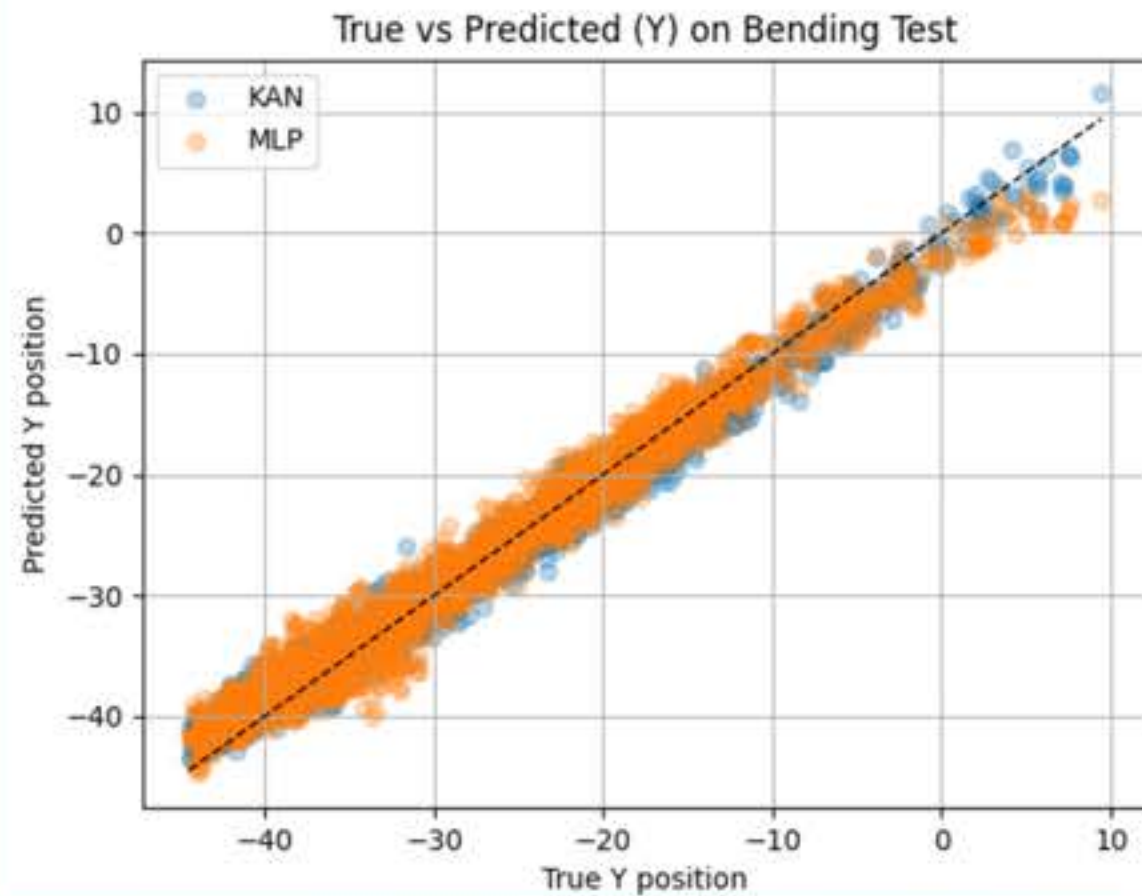
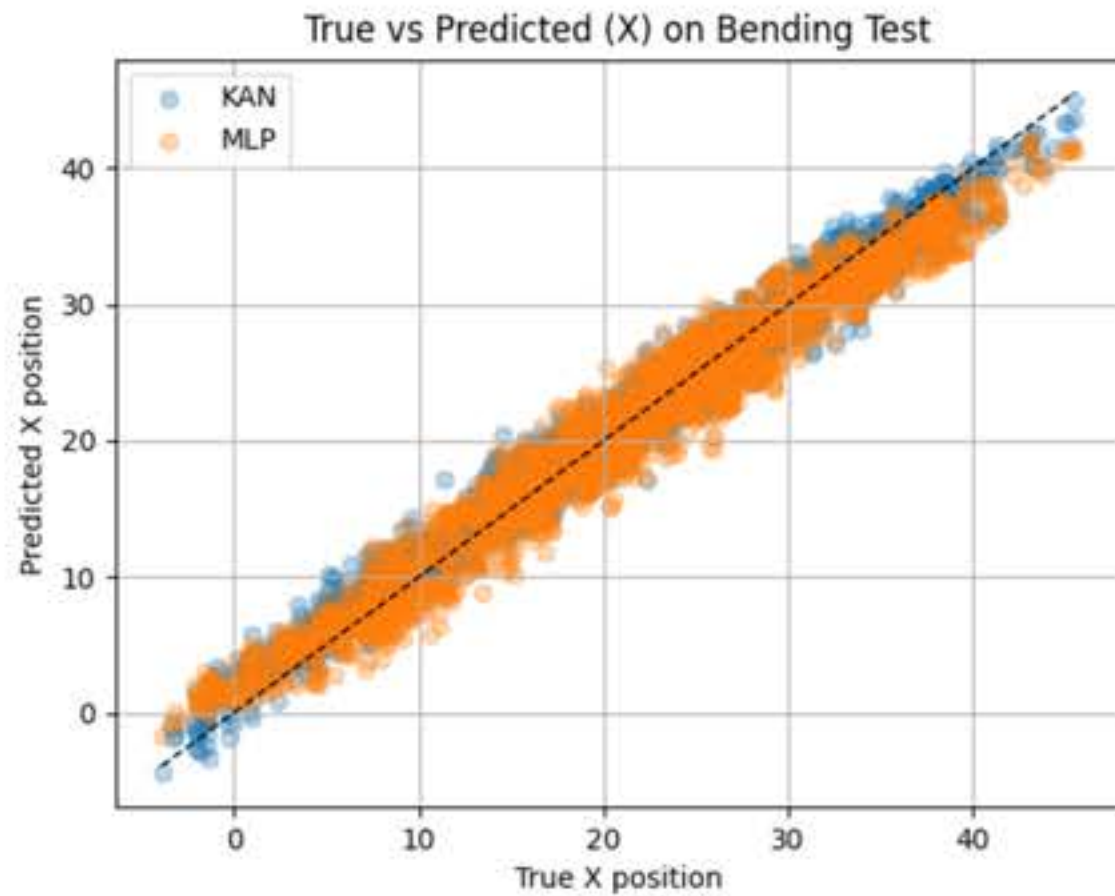




# FIRST LOOK

**(Ms, Ds)**

Overall, both models closely follow the ground truth. KAN is less precise on Z.

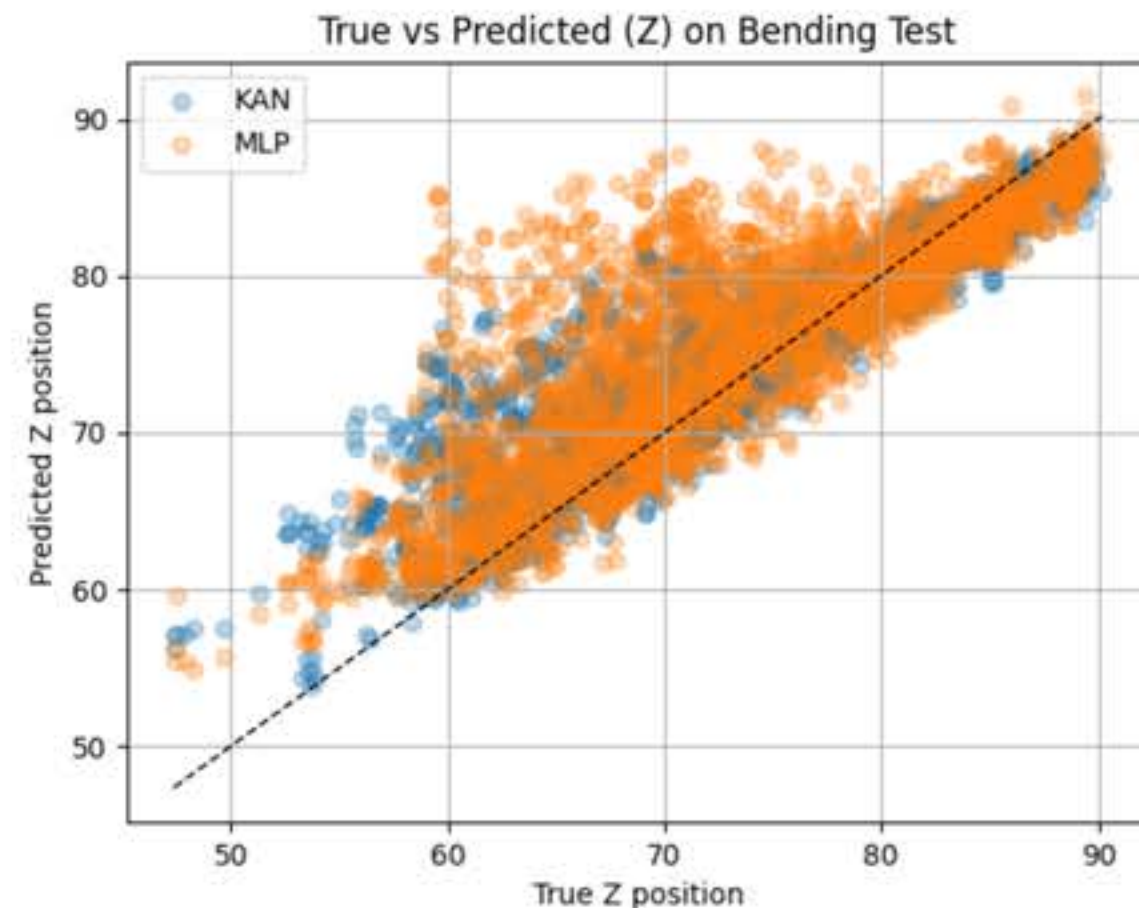
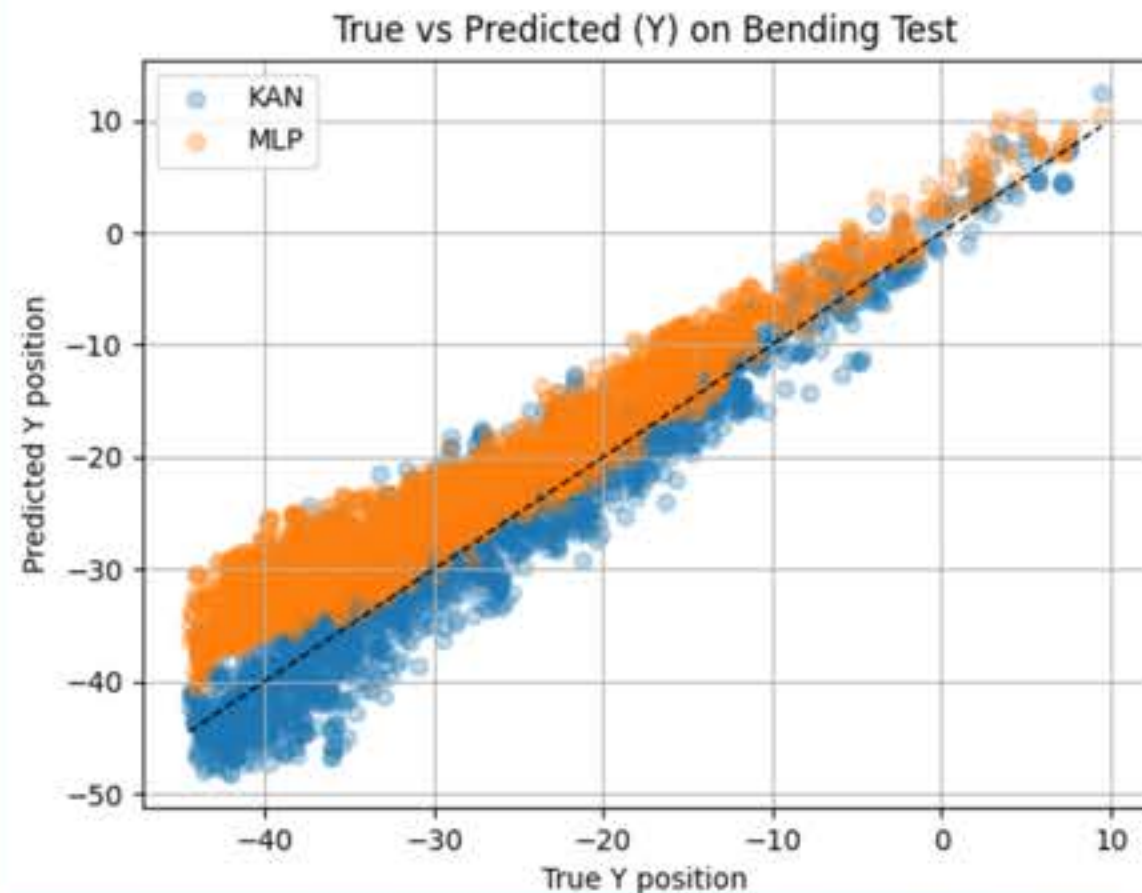
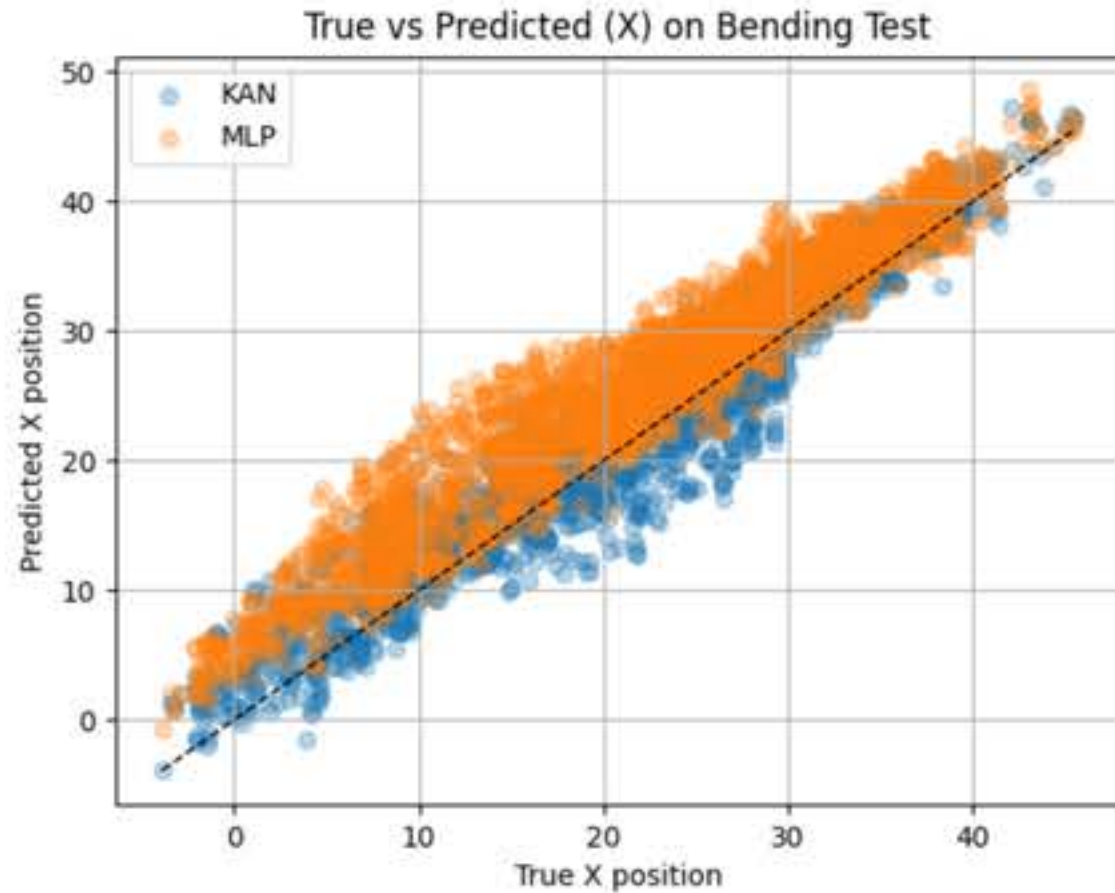




# FIRST LOOK

**(Md, Ds)**

Here KAN demonstrates more consistent and accurate predictions across all axes, while MLP tends to systematically overestimate, particularly along the X and Y directions.

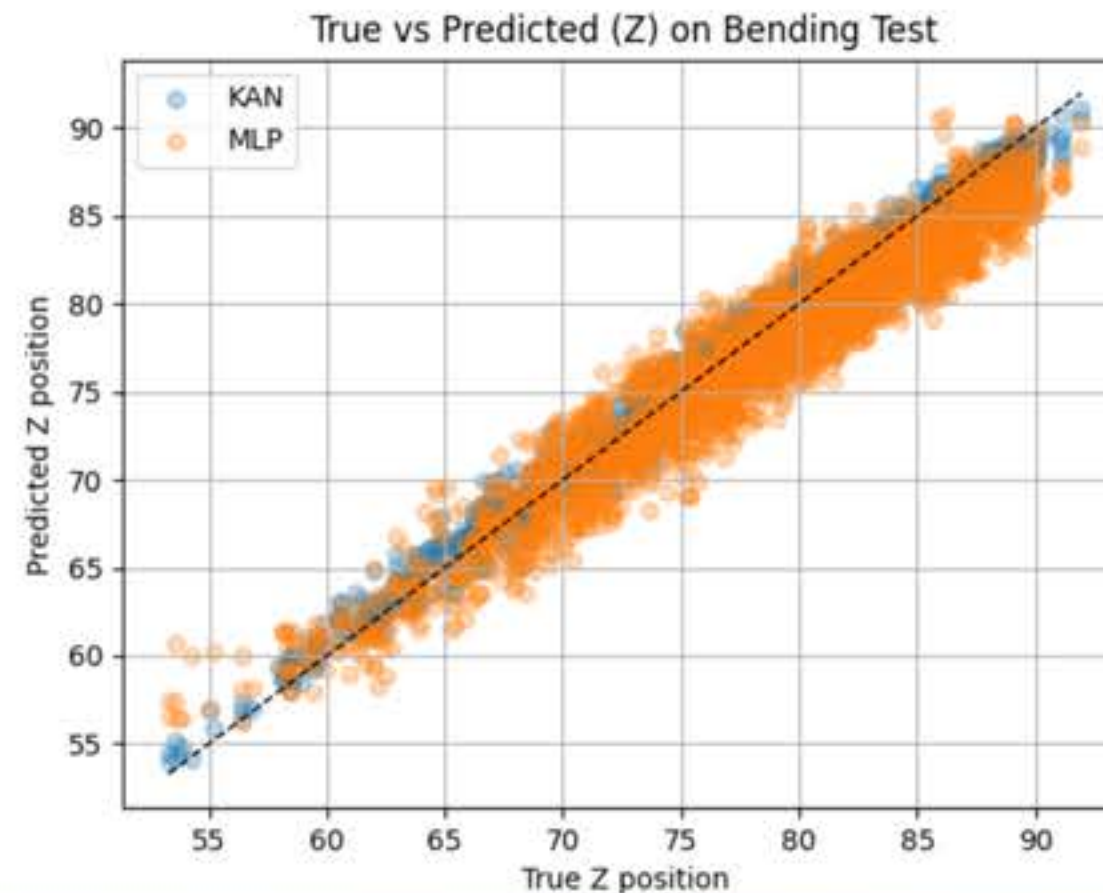
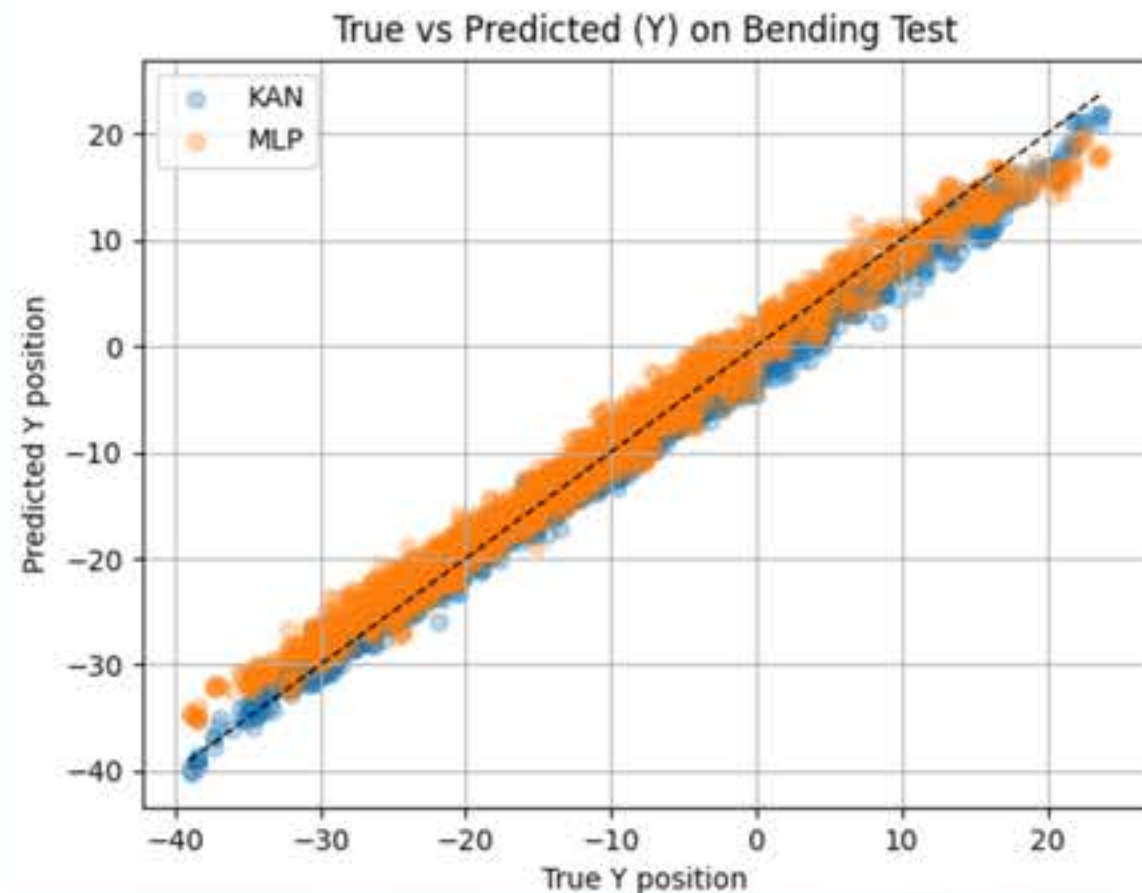
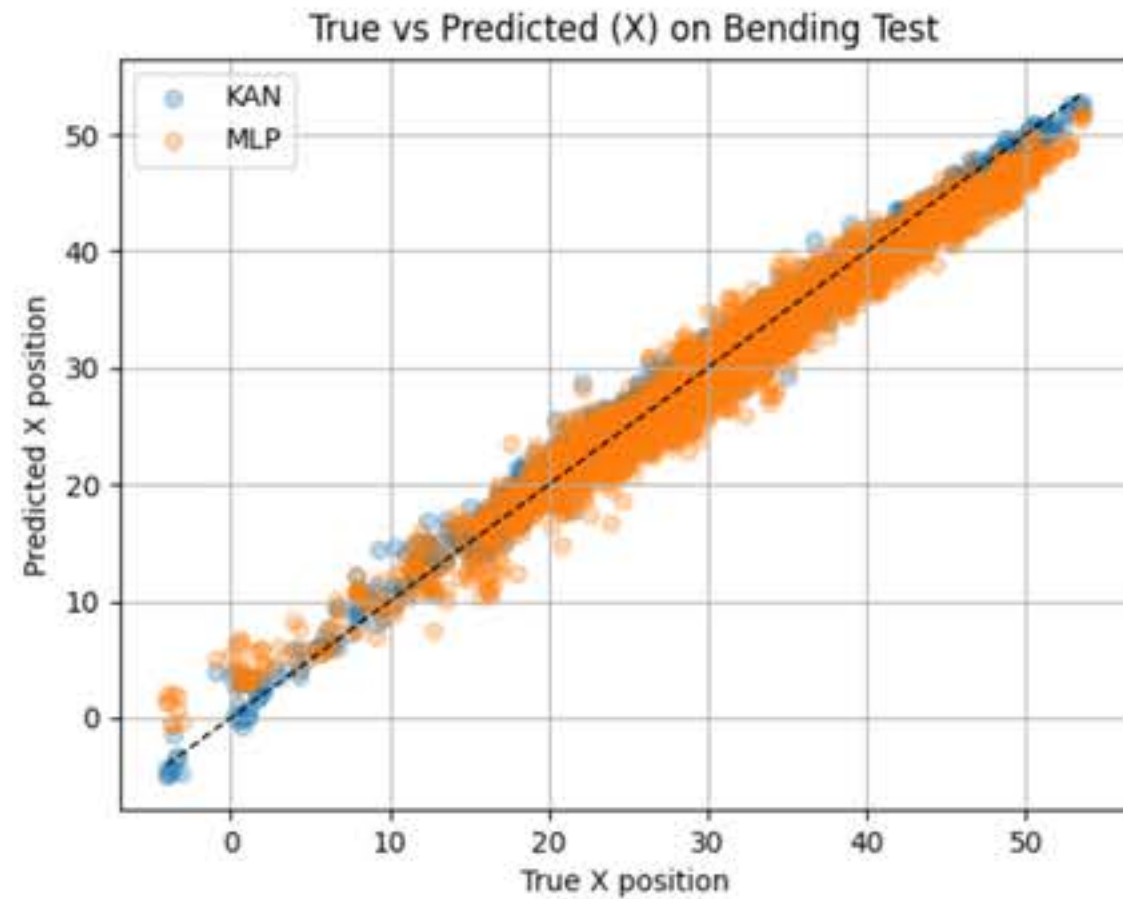




# FIRST LOOK

**(Md, Dd)**

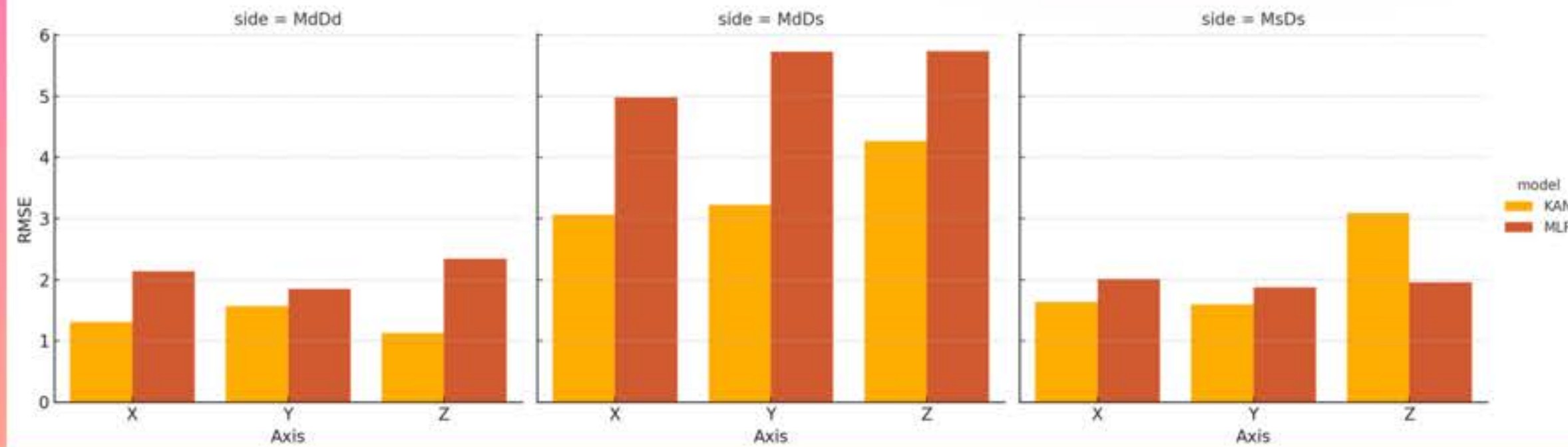
Both KAN and MLP models demonstrate high prediction accuracy across all spatial axes during bending tests, with KAN showing slightly better alignment with the ground truth in the X and Y axes, while both models are comparably accurate along the Z axis.



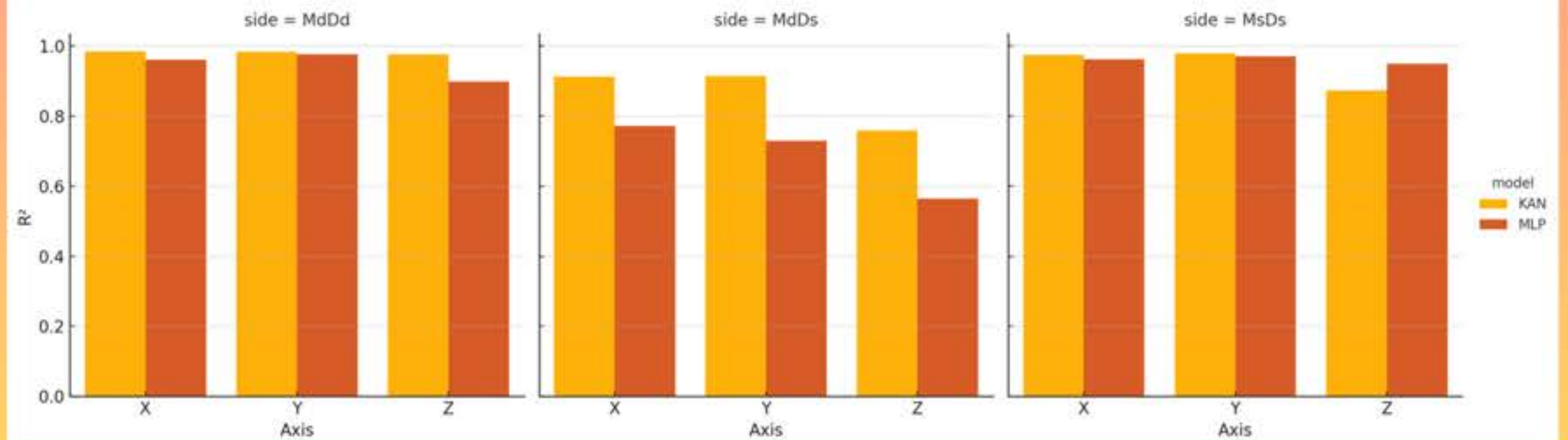


# RESULTS

Confronto RMSE per asse e modello su ogni esperimento



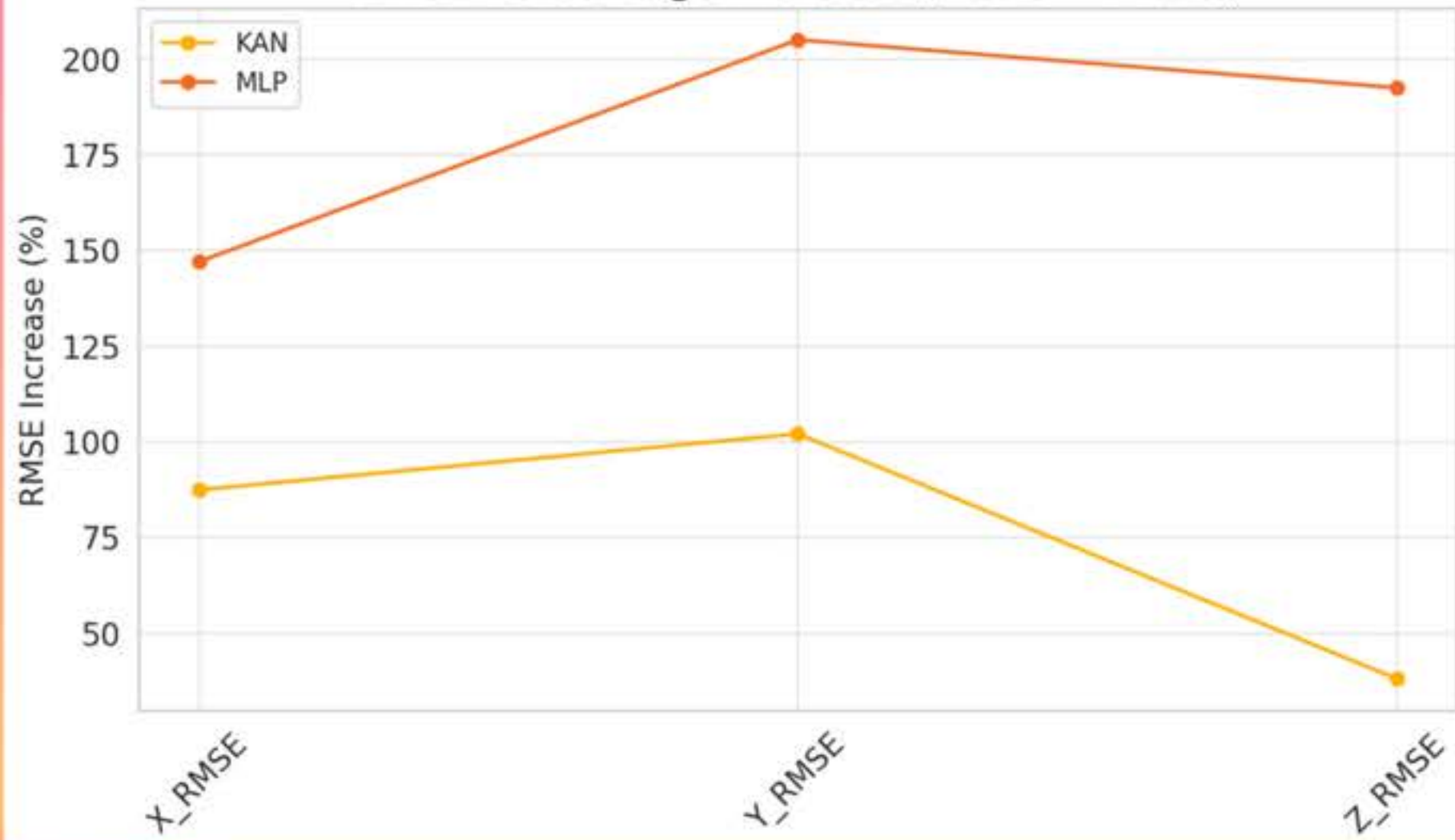
Confronto  $R^2$  per asse e modello su ogni esperimento



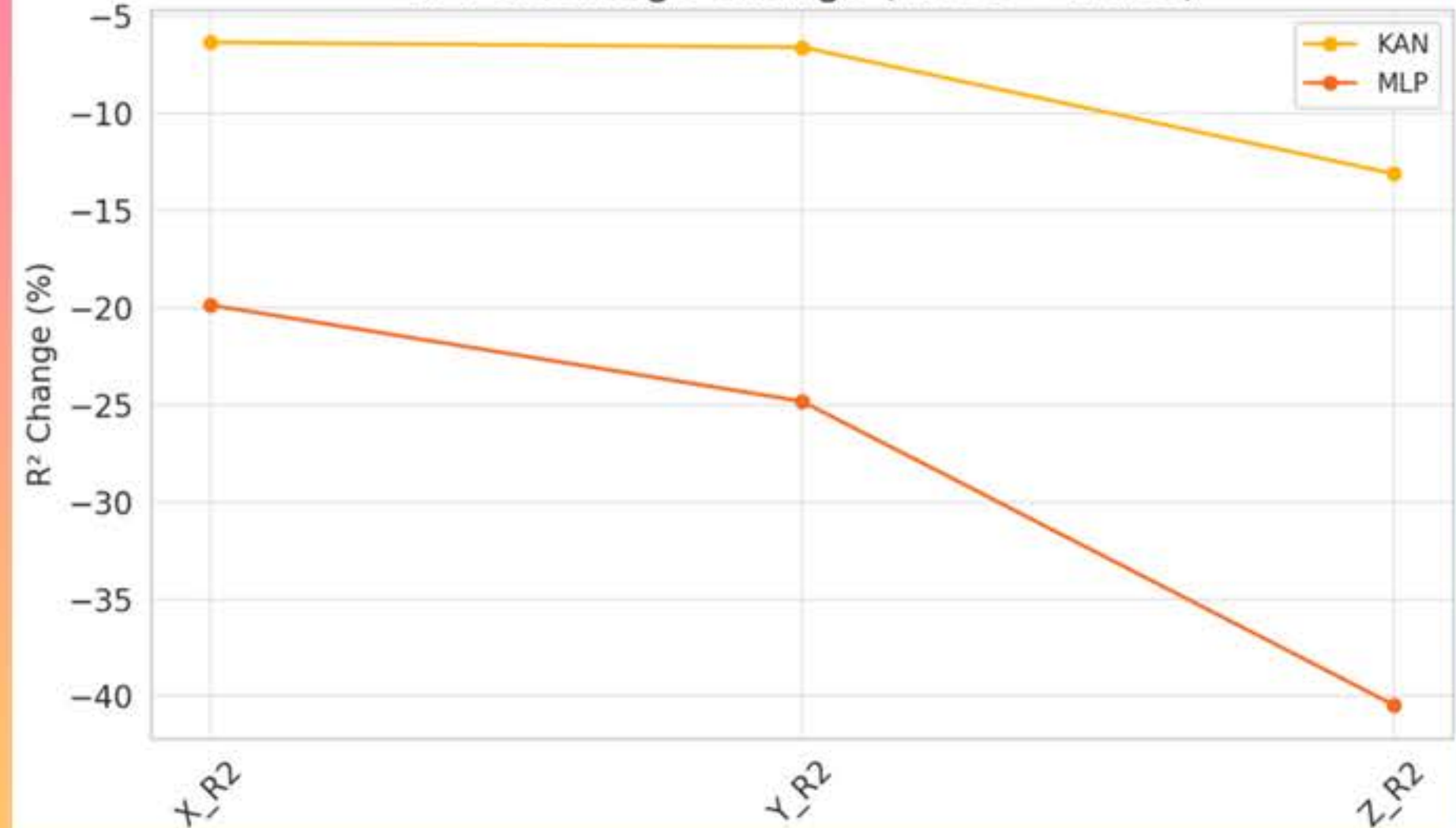


# RESULTS

RMSE Percentage Increase (MsDs → MdDs)



R<sup>2</sup> Percentage Change (MsDs → MdDs)







# INVERSE KINEMATIC EVALUATION



# INVERSE MODEL EVALUATION PIPELINE

To evaluate the performance of inverse models in generating actuator commands for desired workspace trajectories, I implemented and tested two complementary evaluation modes:

1. **Inverse Control Mode (position  $\rightarrow$  actuation)**: In this setup, I designed two synthetic trajectories in the operational space, a *circle* and a *figure-eight* ( $\infty$ ). These trajectories represent *desired end-effector motions*. I then fed the trajectory points (positions) as input to the trained inverse models (KAN and MLP), which predicted the corresponding actuation signals required to follow the path. The predicted actuations were visualized over time to assess their temporal behaviour.
2. **Virtual Robot Mode (actuation  $\rightarrow$  position)**: To verify the actual trajectory generated by the predicted actuations, I also trained a *forward model* that maps actuation inputs back to positions in the workspace. The predicted actuations from the inverse models were passed through this *virtual robot* (the forward model), and the resulting trajectories were compared with the original target paths. This comparison was performed visually, through temporally colour-coded plots of the trajectories—and quantitatively.

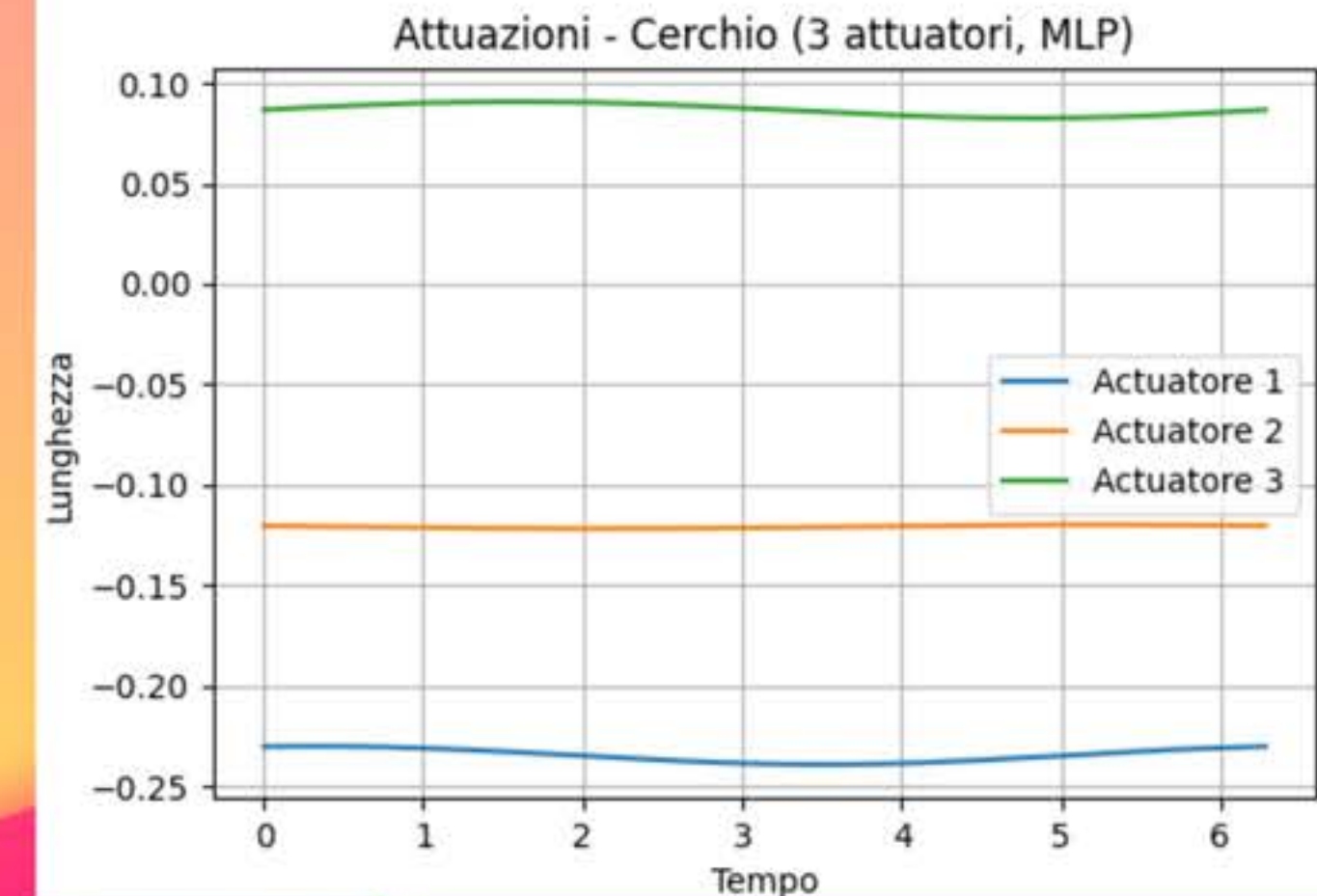
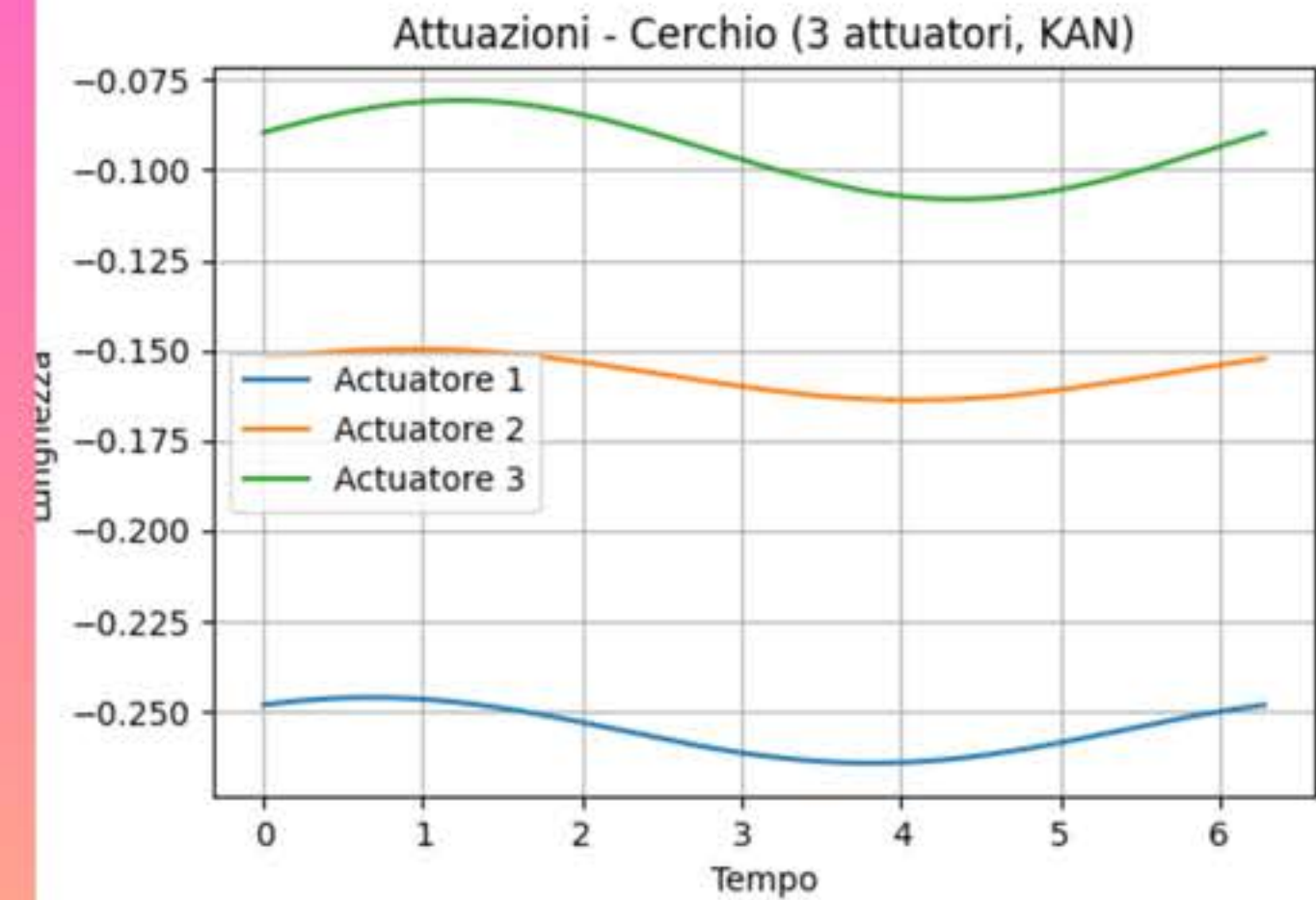


# ACTUATORS

## CICLE TRAJECTORY

The KAN model generates smooth, **sinusoidal-like** actuator signals with clear phase shifts between actuators, reflecting the rotational nature of the circular trajectory. This indicates that the model has *successfully learned* a coordinated and dynamic control strategy.

In contrast, the MLP model produces flatter, **less dynamic** signals with minimal variation over time. This suggests a *lack of sensitivity* to the trajectory's shape and a reduced ability to encode temporal coordination among actuators.



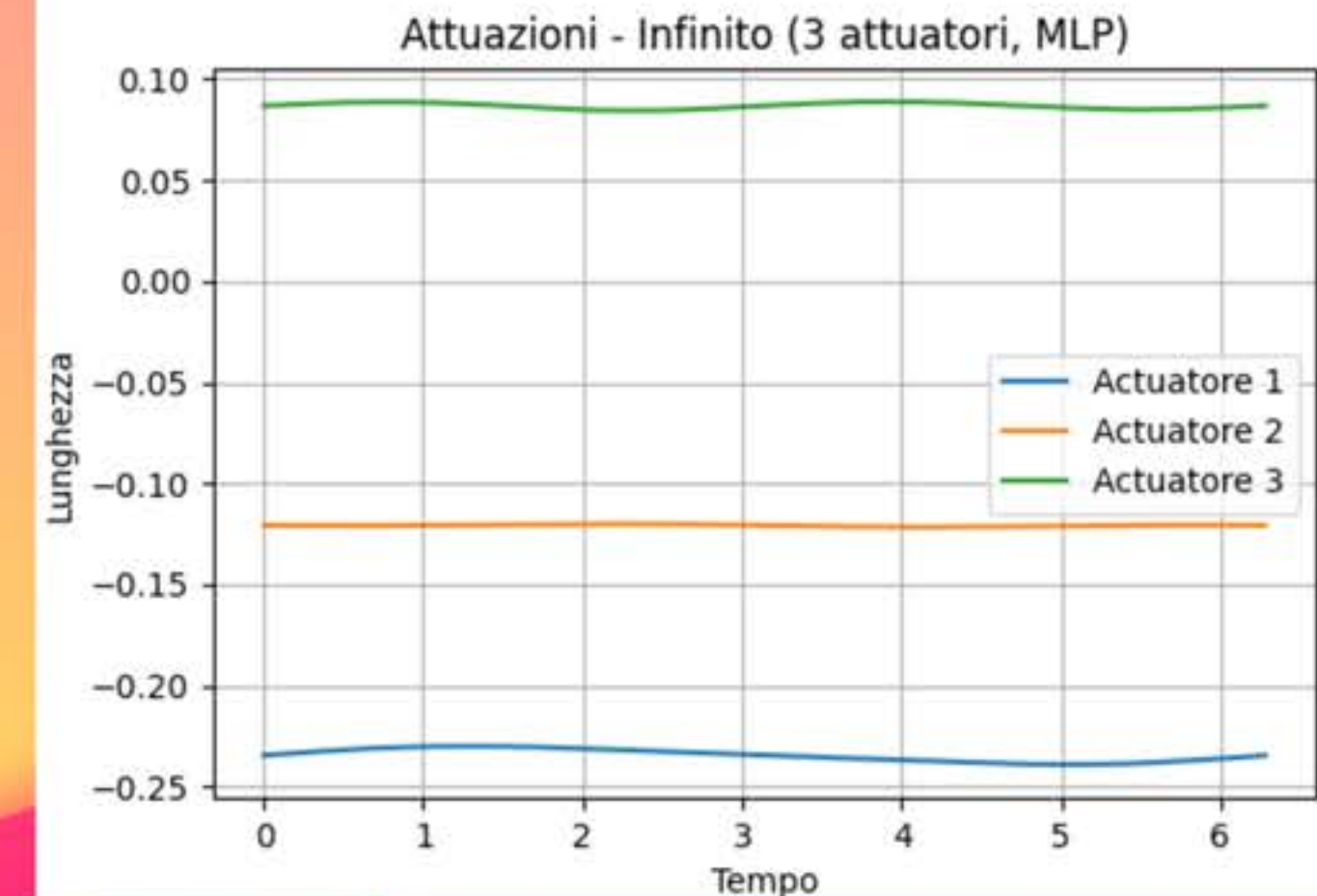
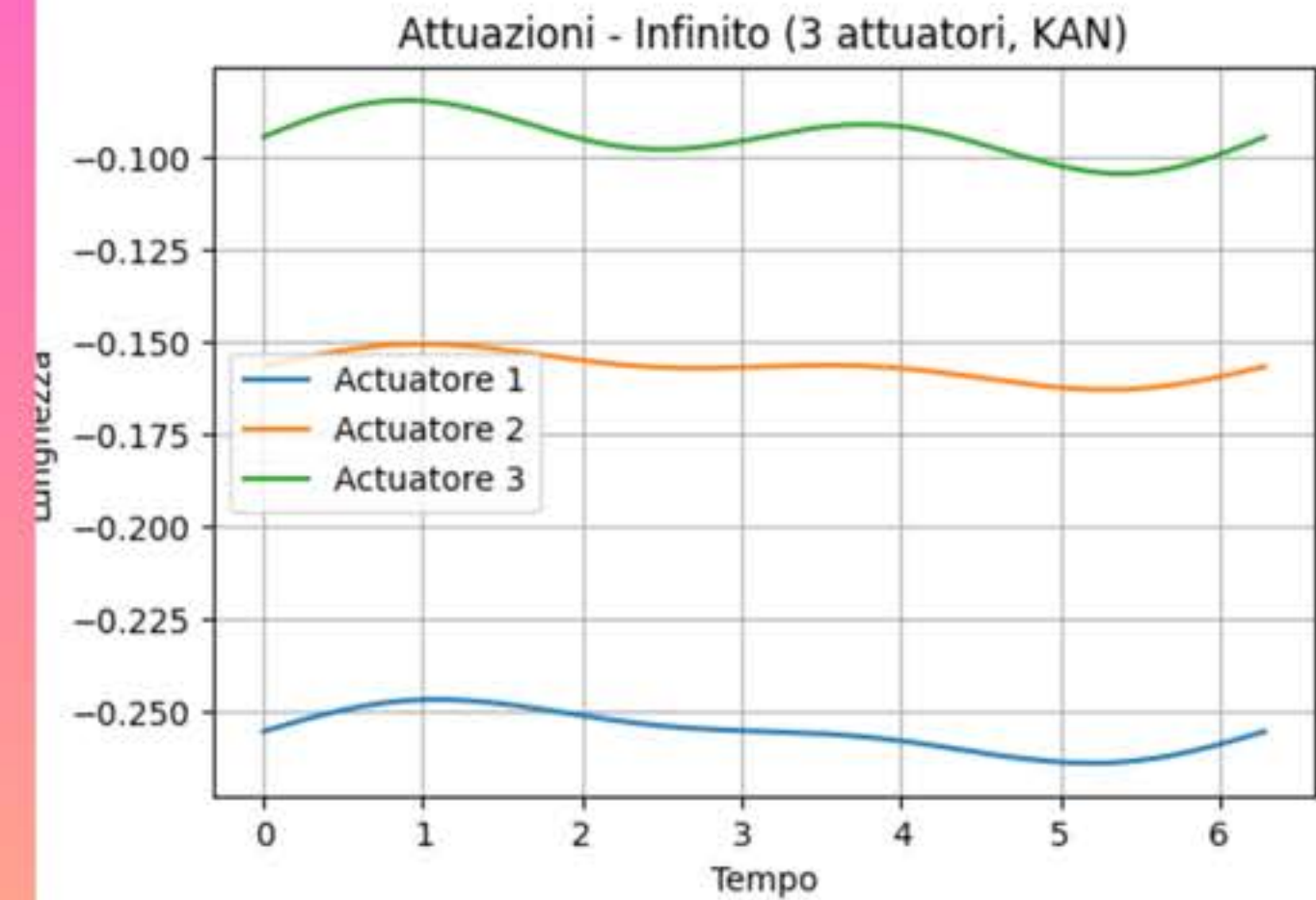


# ACTUATORS

## INFINITY TRAJECTORY

The KAN model again demonstrates **oscillatory actuator behaviour**, adapting to the more complex shape of the figure-eight trajectory. The presence of multiple frequency components in the signals reflects the *non-uniform curvature* and *directional changes* of the path.

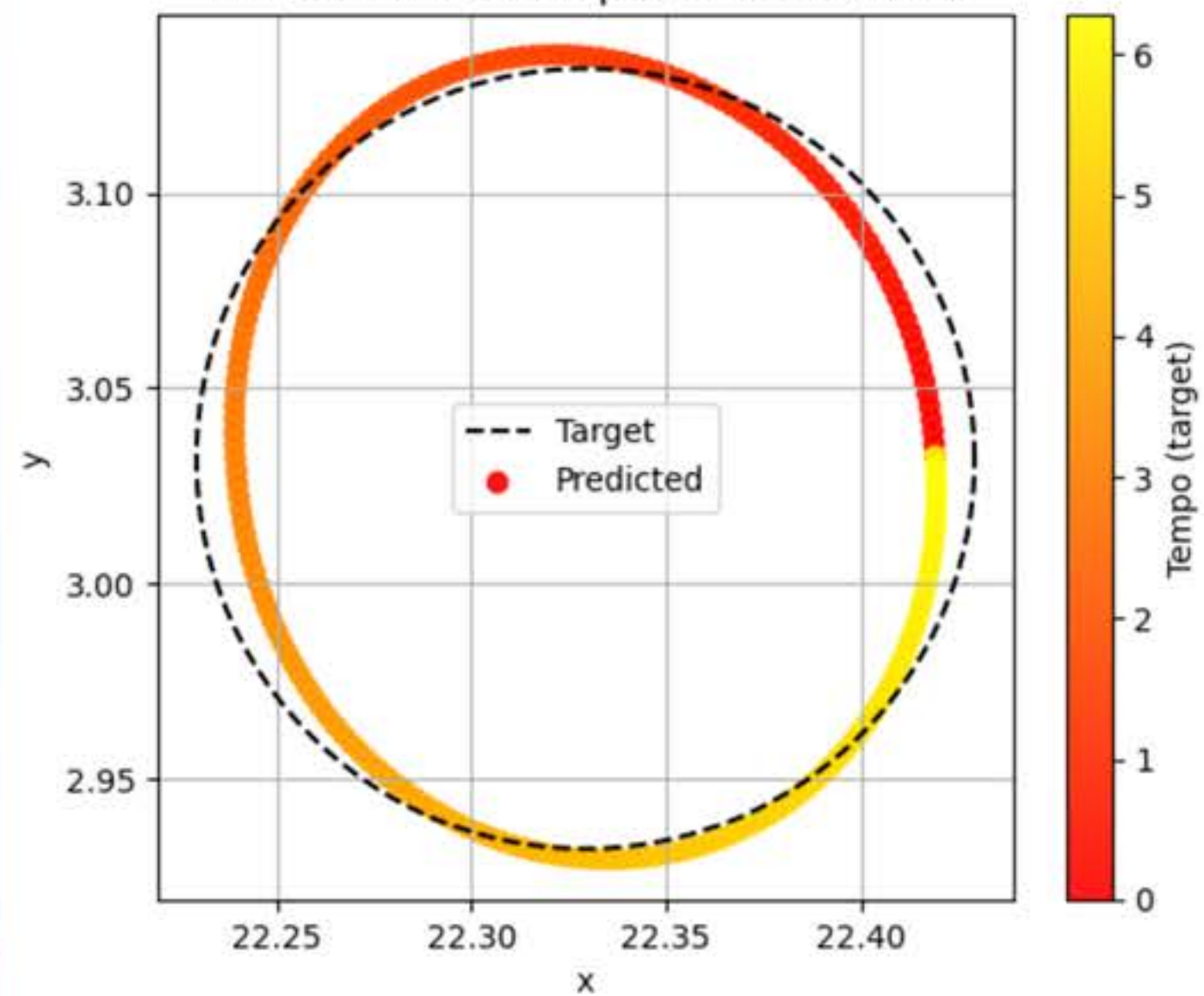
The MLP model, however, shows **limited variation across time**, with signals remaining nearly constant. This implies that the model *fails to capture the trajectory dynamics* and results in less expressive actuation profiles.



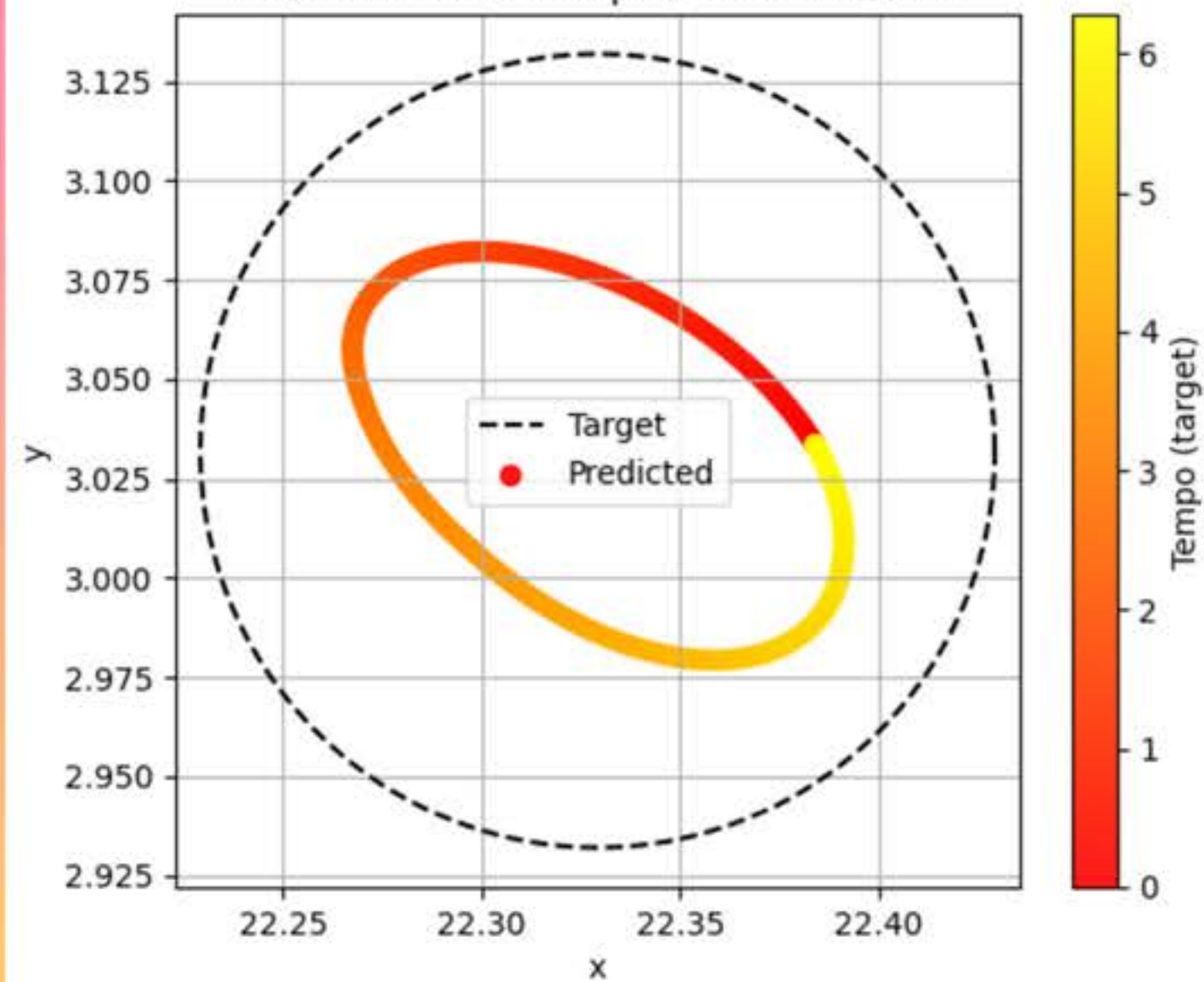


# RESULTS

Traiettoria nel workspace KAN - Cerchio



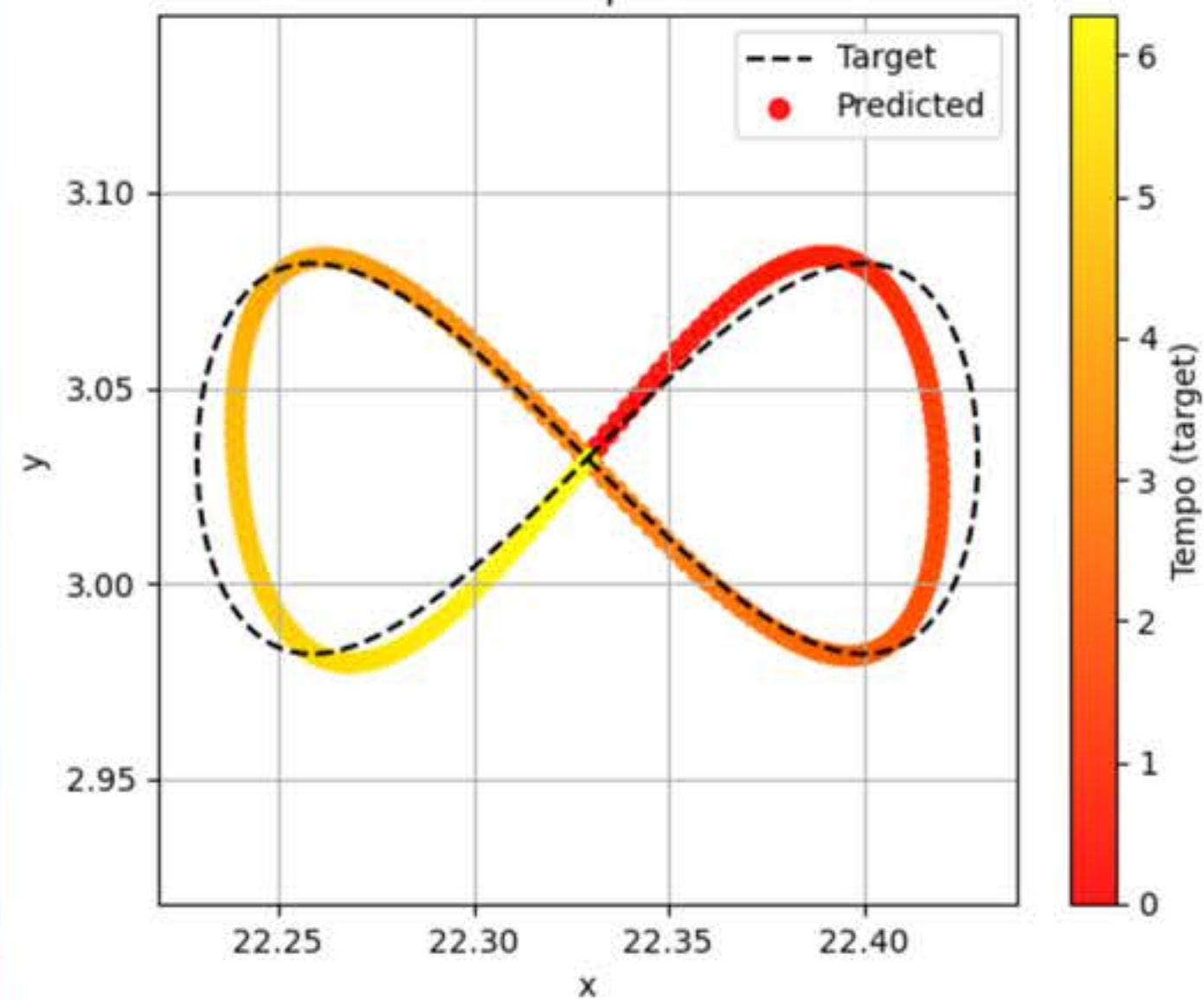
Traiettoria nel workspace MLP - Cerchio



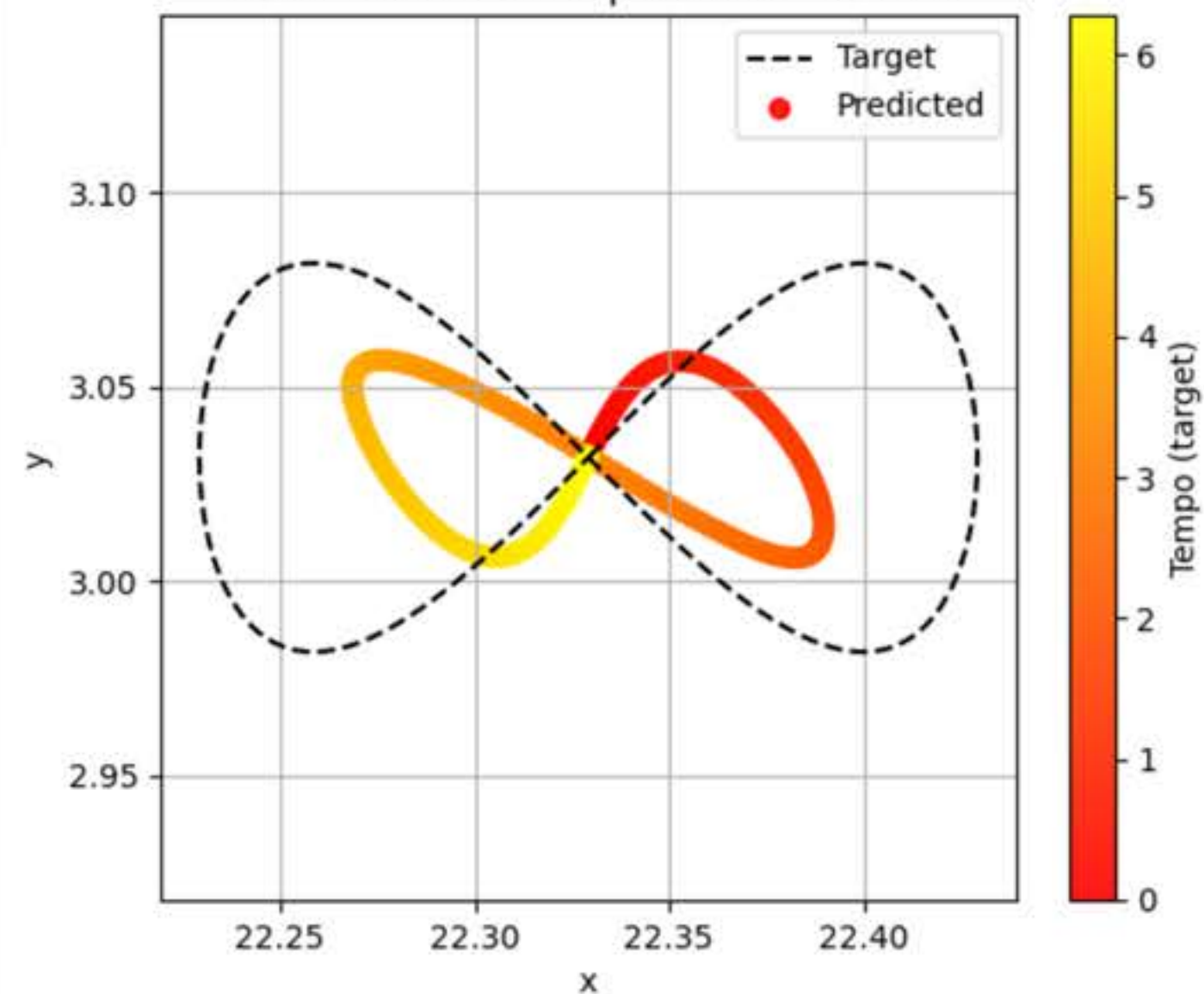


# RESULTS

Traiettoria nel workspace KAN - Infinito



Traiettoria nel workspace MLP - Infinito





# RECURRENT KAN



# ARCHITECTURE

The implementation is based on a custom recurrent neural network architecture that integrates Kolmogorov–Arnold Networks (KANs) to model dynamic and **time-dependent behavior**. By embedding KAN-based transformations within a recurrent structure, the model **captures both nonlinear relationships and temporal dependencies**, making it especially suitable for data-driven modeling of soft robotic systems. The architecture consists of a three-layer recurrent structure that processes sequences of input signals (e.g., actuator commands) over time.

At each time step, the internal state is updated by combining the current input with the previous hidden state using expressive nonlinear transformations. These updates are carried out layer by layer. After processing the entire input, the final state is projected into a low-dimensional space to produce an output prediction, such as the robot tip's position.

This formulation supports both **inverse and forward modeling tasks**, generating actuation signals from desired positions and predicting resulting positions from actuation inputs. Thanks to its recurrent and kernel-based structure, the model is expected to yield smoother and more accurate predictions, improved generalization across different trajectories, and more interpretable internal representations compared to conventional architectures.